OBJECT ORIENTED PROGRAMMING THROUGH JAVA

UNIT1

- **Object Oriented Programming**: Basic concepts, Principles,
- Program Structure in JAVA: Introduction, Writing Simple JAVA Programs, Elements or Tokens in JAVA Programs, JAVA Statements, Command Line Arguments, User Input to Programs, Escape Sequences, Comments, Programming Style.
- **Data Types**, **Variables, and Operators** :Introduction, Declaration of Variables, Data Types, Type Casting, Scope of Variable, Identifier, Literal Constants, Symbolic Constants, Formatted Output with printf() Method, Static Variables and Methods, Attribute Final,
- **Introduction to Operators**, Precedence and Associativity of Operators, Assignment Operator (=), Basic Arithmetic Operators, Increment (++) and Decrement (- -) Operators, Ternary Operator, Relational Operators, Boolean Logical Operators, Bitwise Logical Operators.
- **Control Statements** :Introduction, if Expression, Nested if Expressions, if–else Expressions, Ternary Operator?:, Switch Statement, Iteration Statements, while Expression, do–while Loop, for Loop, Nested for Loop, For–Each for Loop, Break Statement, Continue Statement.

JAVA FUNDAMENTALS

1. OBJECT ORIENTED PROGRAMMING (OOP)

A. Basic Concepts
- Class
- Object
- Method
- Message Passing

B. Principles
- Encapsulation
- Inheritance
- Polymorphism
- Abstraction

C. Advantages of OOP
- Modularity
- Reusability
- Maintainability
- Flexibility

2. PROGRAM STRUCTURE IN JAVA

1. Introduction to Java
2. Writing Simple Java Programs
3. Elements / Tokens in Java Programs
   - Keywords
   - Identifiers
   - Literals
   - Operators
   - Separators
4. Java Statements
5. Command Line Arguments
6. User Input to Programs
7. Escape Sequences
8. Comments
   - Single Line (//)
   - Multi Line (/* */)
   - Documentation (/** */)
9. Programming Style
   - Indentation
   - Naming Conventions
   - Readability
   - Documentation

3. DATA TYPES, VARIABLES AND OPERATORS

A. Data Types

1. Primitive (8 Types)
- byte
- short
- int
- long
- float
- double
- char
- boolean

2. Non-Primitive (Reference Types)
- String
- Array
- Class
- Interface

B. Variables
- Declaration
- Scope of Variable
- Type Casting
- Static Variables
- Static Methods
- final Attribute

C. Constants
- Literal Constants (e.g., 10, 3.14, 'A')
- Symbolic Constants (using final keyword)

D. Formatted Output
printf() Method

E. Operators
1. Assignment (=)
2. Arithmetic (+, −, *, /, %)
3. Increment (++)
4. Decrement (−−)
5. Ternary (? :)
6. Relational (<, >, <=, >=, ==, !=)
7. Boolean Logical (&&, ||, !)
8. Bitwise Logical (&, |, ^, ~, <<, >>, >>>)
9. Precedence & Associativity (of Operators)

4. CONTROL STATEMENTS

A. Decision Making
- if Expression
- Nested if Expressions
- if–else Expressions
- Ternary Operator (? :)
- Switch Statement

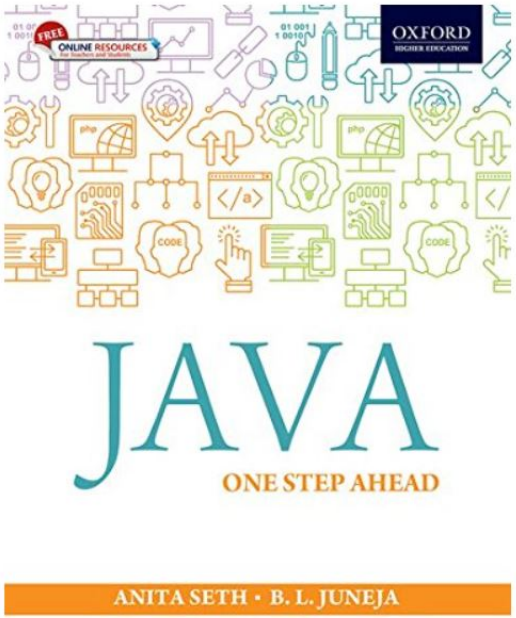
B. Iteration / Looping
- while Expression
- do–while Loop
- for Loop
- Nested for Loop
- For–Each for Loop

C. Jump Statements
- break Statement
- continue Statement

These topics form the foundation of Java Programming and are essential for building problem-solving skills using the Java language.

Text Books

- JAVA one step ahead, Anitha Seth, B.L.Juneja, Oxford.
- Joy with JAVA, Fundamentals of Object Oriented Programming, Debasis Samanta, Monalisa Sarma, Cambridge, 2(



History



- **1990** – Sun Microsystems initiated the **Stealth Project**, later known as the **Green Project**.
- **1991** – The Green Project team was formed, consisting of **James Gosling, Patrick Naughton, Mike Sheridan, and Bill Joy**.
- **1991** – **James Gosling** began developing a new programming language inspired by **C and C++**, with the primary goal of creating a **platform-independent** language.
- The language was initially named **Oak**, after an oak tree outside Gosling's office.
- Since **Oak** was already a registered trademark, it was later renamed **Java**.
- **1994** – **Patrick Naughton** developed **WebRunner**, a Java-based web browser to demonstrate the capabilities of **Java applets**.
- It was later renamed **HotJava**.
- **1995** – **Sun Microsystems** officially announced **Java** and the **HotJava** web browser, marking Java's public release..

Processor Developers

- *Intel*
- *AMD*
- *Qualcomm*
- *NVIDIA*
- *IBM*
- *Samsung*
- *Motorola*
- *Hewlett-Packard (hp)*
- *Dell*
- *Acer*
- *VIA*
- *Marvell*
- *Global Foundries*
- *Media Tek*
- *Atmel*
- *Freescale*
- *MSNCC-Microcell*
- *Transmeta*
- *Cyrix*
- *Rockchip*

C

Operating System	Processor	Same .exe Runs?
Windows	Intel	✔
Windows	ARM	✘ (Recompile Required)
Linux	Intel	✘ (Recompile Required)
Linux	ARM	✘ (Recompile Required)
macOS	Apple M-Series	✘ (Recompile Required)

JAVA

Operating System	Processor	Runs?
Windows	Intel	✔
Windows	ARM	✔
Linux	Intel	✔
Linux	ARM	✔
macOS	Apple M-Series	✔

History of Java

- JDK Alpha and Beta (1995)
- JDK 1.0 (23rd Jan, 1996)
- JDK 1.1 (19th Feb, 1997)
- J2SE 1.2 (8th Dec, 1998)
- J2SE 1.3 (8th May, 2000)
- J2SE 1.4 (6th Feb, 2002)
- J2SE 5.0 (30th Sep, 2004)
- Java SE 6 (11th Dec, 2006)
- Java SE 7 (28th July, 2011)
- Java SE 8 (18th March, 2014) - 217 Packages and 4240 Classes.
- Java SE 9 (September 2017)
- Java SE 10 (March 2018)
- Java SE 11 (September 2018)
- Java SE 12 (March 2019)
- Java SE 13 (September 2019)
- And so on at present JAVA SE 26 (March 17, 2026)

Why Java populated

Portability
Security
Applets

About Java

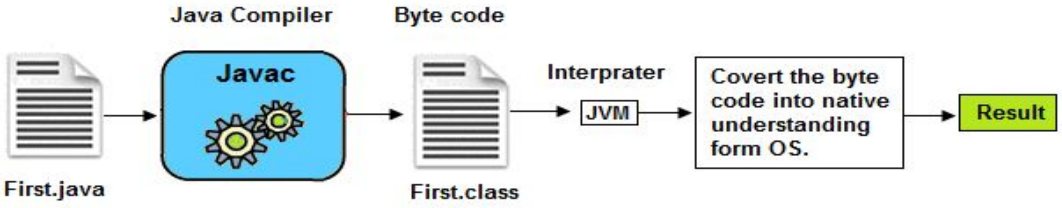
Platform	Full Form	Purpose
Java SE (formerly J2SE)	Java Platform, Standard Edition	Used for developing and deploying standalone desktop and console applications. It provides the core Java libraries and APIs.
Java EE (formerly J2EE, now Jakarta EE)	Java Platform, Enterprise Edition	Used for developing large-scale, distributed, and web-based enterprise applications such as e-commerce, banking, and ERP systems.
Java ME (formerly J2ME)	Java Platform, Micro Edition	Designed for developing Java applications for embedded systems and resource-constrained devices such as feature phones, IoT devices, and consumer electronics.

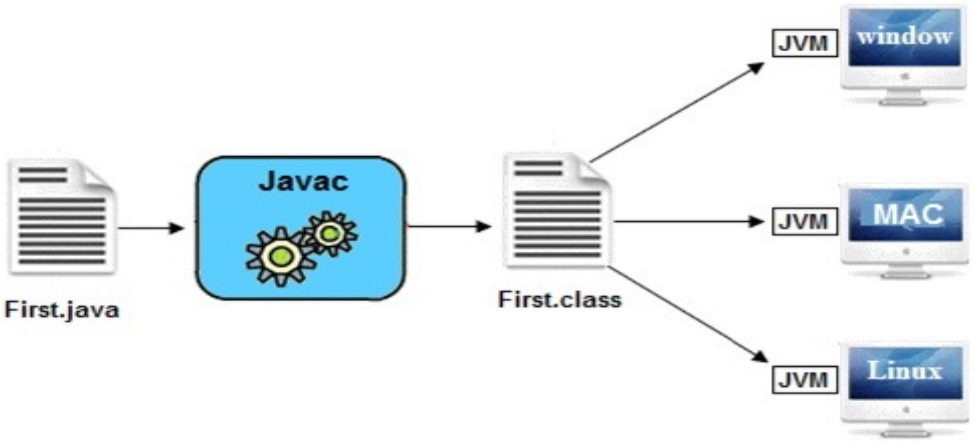
Java SE

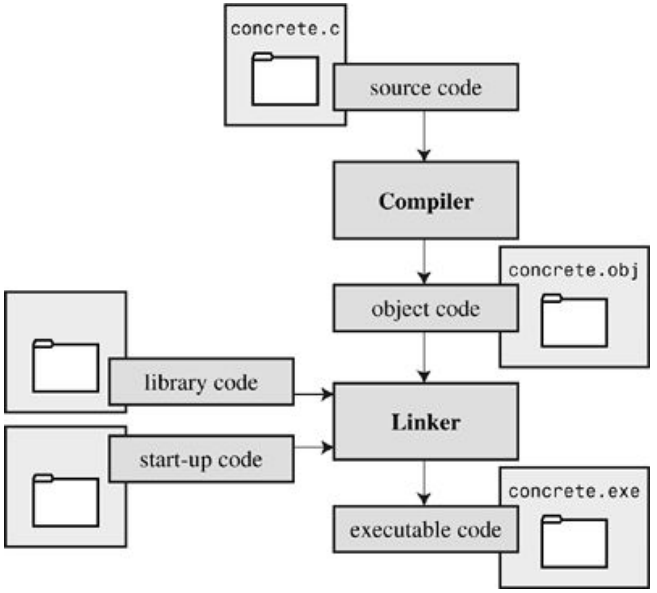
- Total Java Classes
  - 4240 in java 8
  - 4024 in java 7
  - 3793 in java 6
  - 3279 in java 5.0
- Total Java Packages
  - 217

As per SE22
Total Classes 5000+
Total Packages 430

Java's Magic : Bytecode



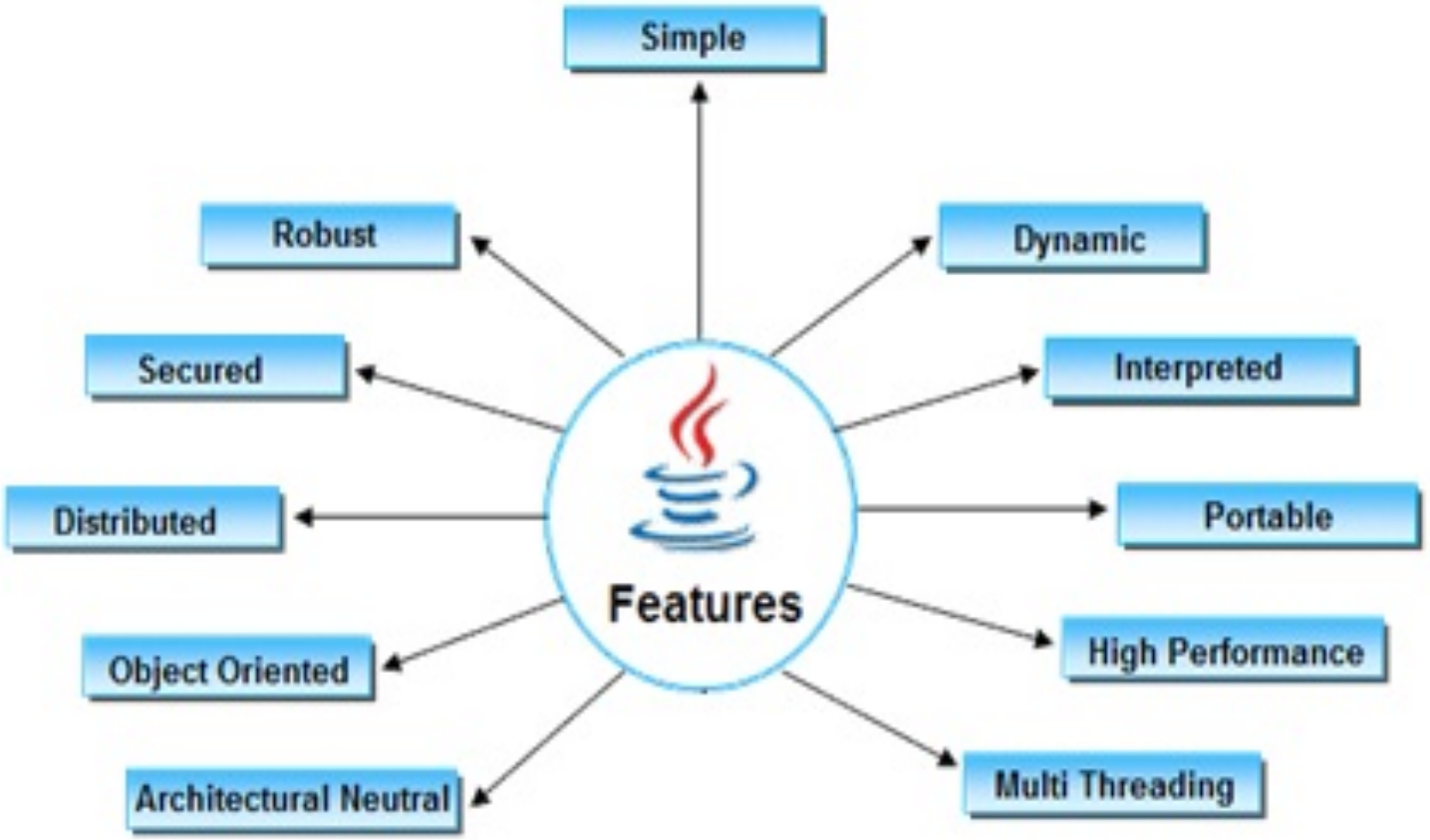




Byte code: optimized set of instructions

JIT (Just in Time Compiler)

Java Features



Java Features

- **Simple**
  - Java has a simple syntax that is easy to learn.
  - It removes complex features of C++, such as pointers, operator overloading, and multiple inheritance (through classes).
- **Object Oriented**
  - Java is based on the concepts of **Objects** and **Classes**.
  - It supports **Encapsulation, Inheritance, Polymorphism,** and **Abstraction**, making programs modular and reusable.
- **Robust**
  - Java is robust or strong programming language because of its capability to handle run-time errors, such as exception handling.

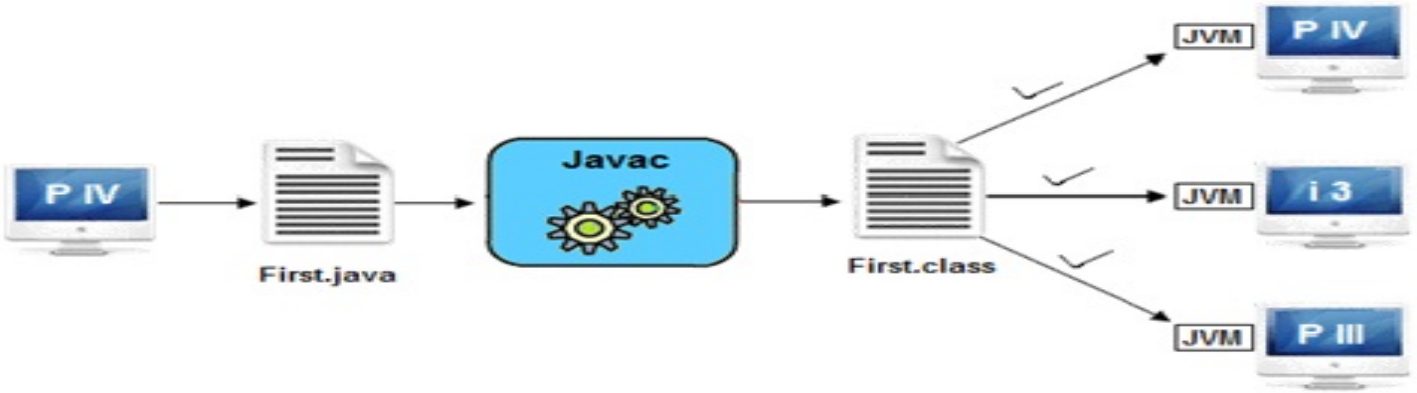
Java Features

- **Multi Threading:**
  - Java supports multiple threads executing simultaneously.
  - This allows efficient utilization of CPU resources and improves application responsiveness.



Java Features

- **Architecture-Neutral: - Hardware Independence**
  - Java source code is compiled into bytecode, which is independent of the underlying hardware architecture.
  - The same bytecode runs on any system that has a



Java Features

- **Interpreted:**
  - Java byte code is interpreted (and JIT-compiled) by the JVM.
  - This enables platform independence.

- **High Performance:**
  - The problem with interpreter inside JVM is that it is slow.
  - Because of this, Java programs used to run slow.
  - To overcome this, along with interpreter, Java people have introduced JIT (Just-in-Time) compiler, which enhances the speed of execution.

Java Features

- **Distributed:**
  - Java supports distributed computing over networks.
  - It provides APIs for networking, Remote Method Invocation (RMI), Web Services, and distributed applications.
- **Dynamic:**
  - Java supports dynamic loading of classes during program execution.
  - New classes and libraries can be loaded whenever required.

Java Features

- **Secure:**
  - It is more secured language compare to other languages; In this, program is covered into byte code after compilation which is not readable by human
  - Restricted to access other areas in the system
- **Portable:** - **Operating System Independence**
  - Java programs can run on different operating systems without modification.
  - This is possible because of the **Java Virtual Machine (JVM)**.
  - If any language supports platform independent and architectural neutral features, then that language is portable.

Platform Independence =

Hardware Independence (architectural Neutral)

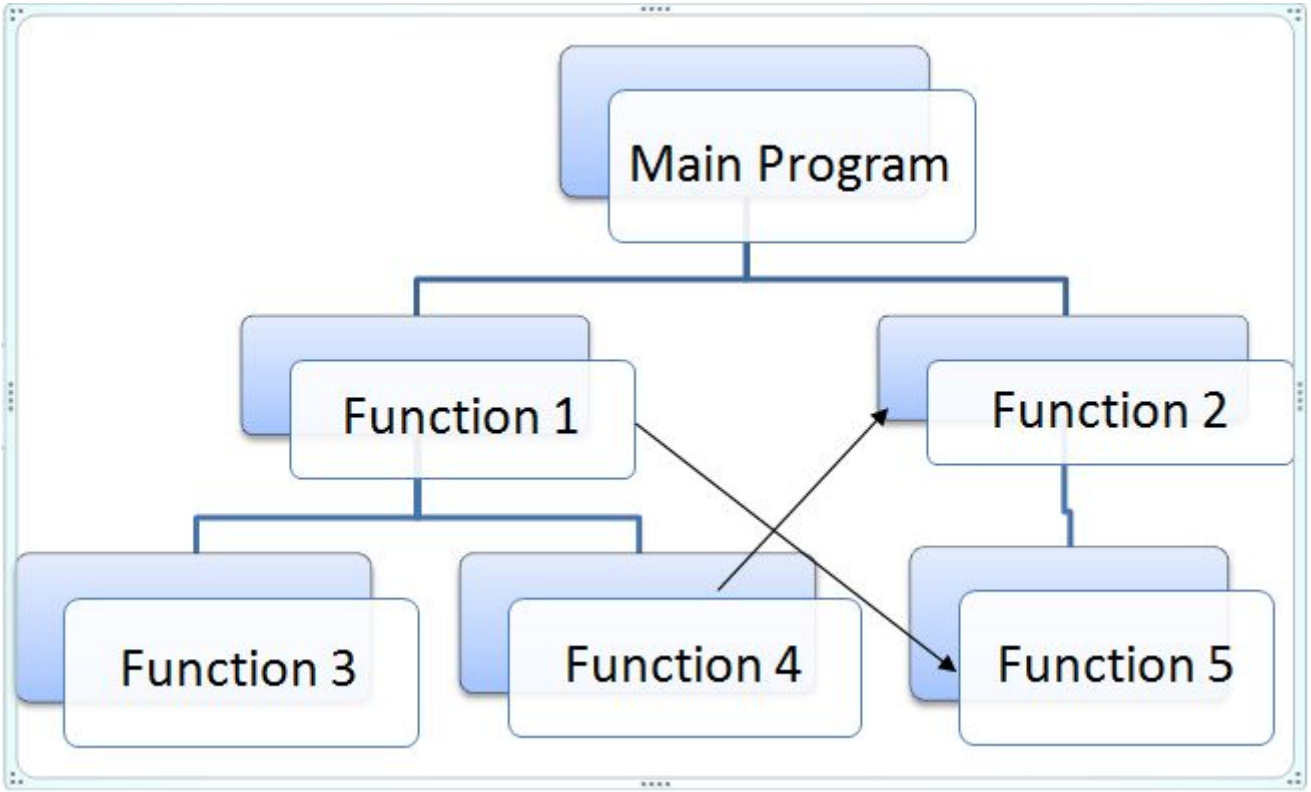
+

Portable (Operating System Independence through JVM)

Procedural/Structured Programming

- Huge programs are divided into smaller programs as functions. These functions can call one another.
- No importance is given to security of data and importance is laid on doing things.
- Data passes globally from function to function
- Most functions access global data
- Follows top-to-bottom approach

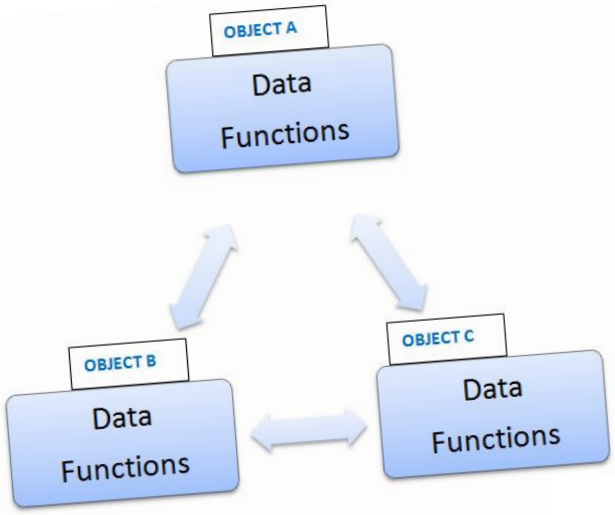
Procedural/Structured Programming



Object Oriented Programming

- *OOP* has more importance to data rather than doing things
- Programs are divided into classes .
- OOP allows bottom-up approach
- New data items and member functions can be comfortably added whenever essential.
- Data is private and prevented from external functions
- Objects can communicate with each other through methods

Object Oriented Programming



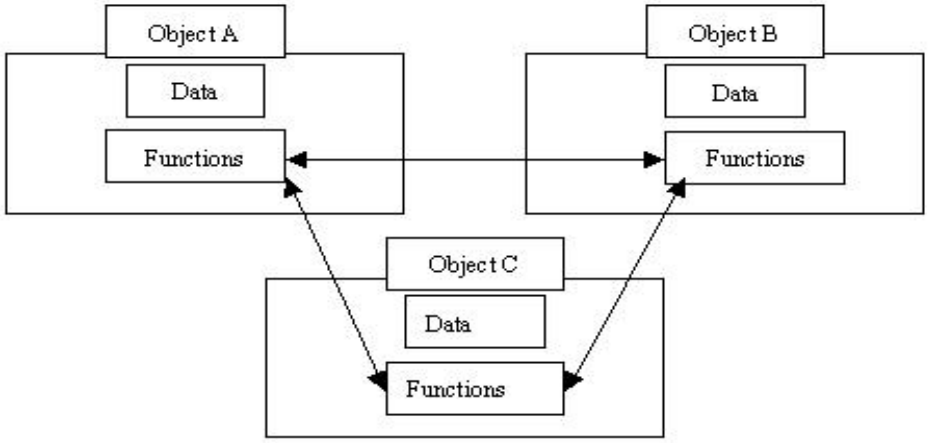


Fig 4

Difference Between POP and OOP

POP (Procedure-Oriented Programming)	FEATURE	OOP (Object-Oriented Programming)
Focuses on functions/procedures.	1. Focus	Focuses on objects and classes.
Data and functions are separate. (DATA ↔ FUNCTIONS)	2. Data & Functions	Data and functions are combined into objects. (OBJECT: DATA, FUNCTIONS)
Follows a top-down approach.	3. Approach	Follows a bottom-up approach.
Data is less secure because it is globally accessible.	4. Data Security	Data is more secure through encapsulation.
Does not support inheritance or polymorphism.	5. Key Features	Supports inheritance, polymorphism, encapsulation and abstraction.
Suitable for small programs.	6. Program Size	Suitable for large and complex programs.
Less code reusability and harder to maintain.	7. Code Reusability & Maintenance	More code reusability and easier to maintain.
Easier to write for simple tasks.	8. Ease	Easier to manage and extend in large applications.
Examples: C, Pascal	9. Examples	Examples: Java, C++, Python, C#

IN SIMPLE WORDS

POP:

The program is divided into functions that perform tasks.

Example:

Create separate functions like add(), delete(), and update().

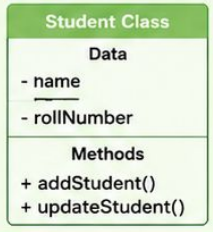


OOP:

The program is divided into objects that contain both data and functions.

Example:

Create a Student class with data (name, roll number) and methods like addStudent() and updateStudent().



Conclusion: POP is function-centered, while OOP is object-centered.

Need of OOP

- Though procedural-oriented programs are extremely powerful, they have some of the following limitations:
  - It is difficult to add any new functionality or to upgrade a program.
  - Functions have unrestricted access to global data.
  - Importance is given to the operation on data rather than the data itself.

Need of OOP

- An object-oriented approach to the application development enables us to design programs.
  - *OOP* has more importance to data rather than doing things
  - Programs are divided into classes - Modularity .
  - OOP allows bottom-up approach
  - New data items and member functions can be comfortably added whenever essential
  - Data is private and prevented from external functions
  - Objects can communicate with each other through methods

Advantages of OOP

- ***Modular structure -*** provides a clear and modular structure for program development.
- ***Easy to maintain and upgrade-*** Object-oriented systems can be easily upgraded.
- ***Inheritance - A***llows us to eliminate redundant code and use existing classes.
- ***Reusability*** - functions and modules can be reused by other users without any modification.
- ***Data hiding -***The programmer can hide the data and functions in a class from other classes.

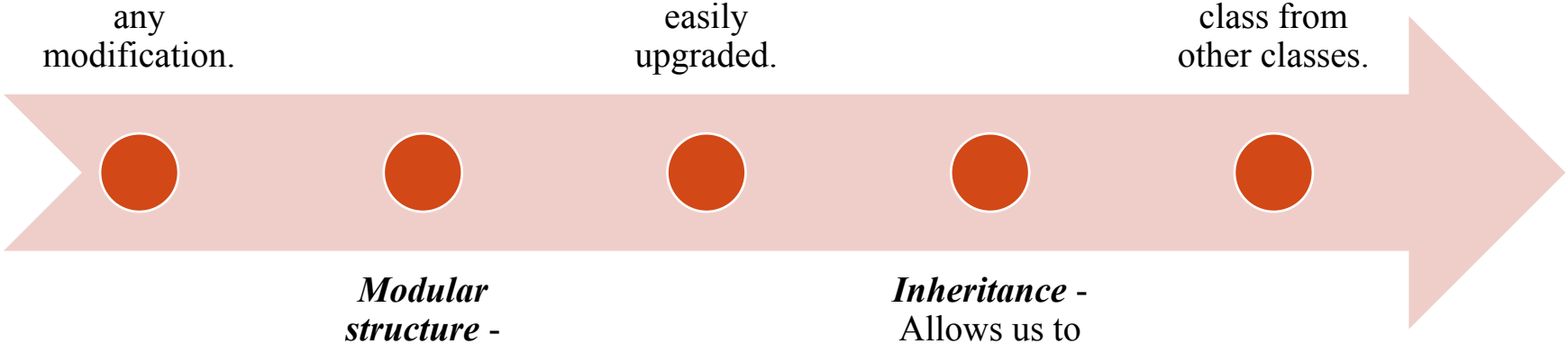
Reusability - functions and modules can be reused by other users without any modification.

Modular structure - provides a clear and modular structure for program development.

Easy to maintain and upgrade- Object-oriented systems can be easily upgraded.

Inheritance - Allows us to eliminate redundant code and use existing classes.

Data hiding -The programmer can hide the data and functions in a class from other classes.



Characteristics of Object-Oriented Programming

- Class
- Object
- Methods and Message Passing
- Data Abstraction
- Encapsulation and Data hiding
- Inheritance
- Polymorphism
- Delegation

Core Pillars of OOP:

- **Abstraction**
- **Encapsulation**
- **Inheritance**
- **Polymorphism**

Class

Object	Properties	Methods
	car.name = Verna car.model = 2017 car.weight = 1250kg car.color = white	car.start() car.drive() car.brake() car.stop()

- In OOP, a **class** is a basic building block.
- A class is a user-defined data type.
- It consists of data members and member functions (methods), which can be accessed and used by creating an instance of that class

A class is a blueprint or template that defines data (variables) and behavior (methods).

Object



- An Object is an instance of a Class.
- When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.
- An object has an identity(name), state/properties, and behavior.

Data Abstraction

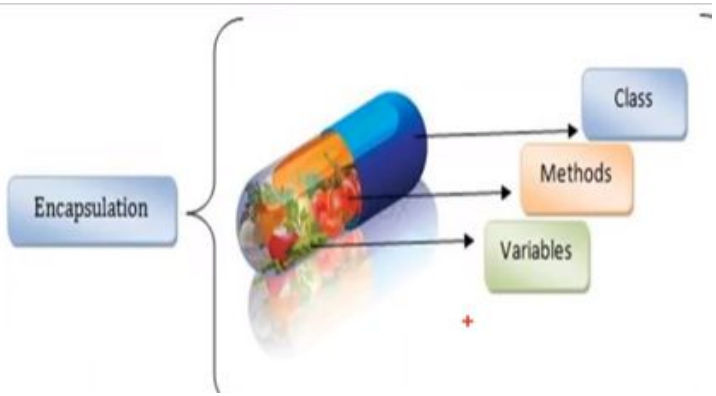


- Abstraction means showing only essential features and hiding implementation details.
- **Example:** When driving a car, you use the steering and pedals without knowing the engine's internal mechanism.

Encapsulation and Data hiding

- It is a mechanism of wrapping the **data and methods** into a single unit (i.e., class).
- The data of a class may be hidden so that it cannot be accessed by the outside code.

  -Ex: Account balance cannot be accessed directly from outside the class.



- Classes have the provision to hide the data or methods from the user.
- A class member may be declared as *public*, *protected* or *private*.
  - A member declared as private cannot be accessed by any code outside the class.
  - Only class members can access a private member.
  - The one declared protected can be accessed by the class members as well as members of the subclass derived from the class.
  - A member declared public can be accessed by any outside code.
  - Whereas the one declared without any access specifier is accessible to classes of the same package only.

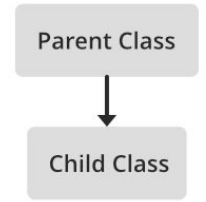
Access Specifiers

- Public
- Private
- Protected
- Default

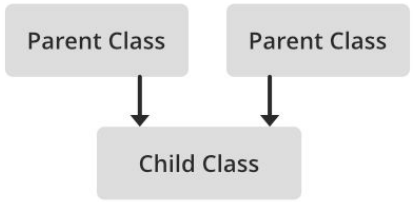
Applied to variables, constructors, methods, and classes.

Inheritance

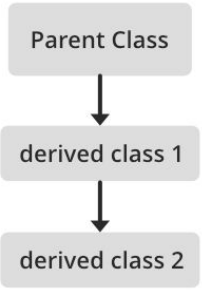
- Inheritance allows one class to acquire properties and methods of another class.
- Allows **reusability**



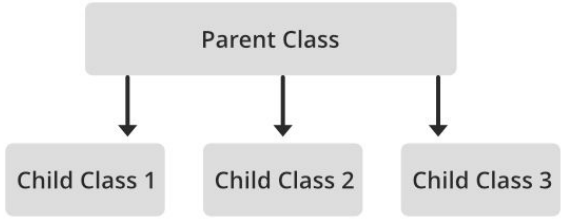
Single inheritance



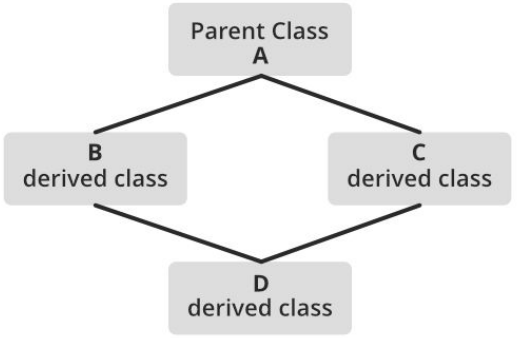
Multiple inheritance



Multilevel inheritance



Hierarchical inheritance



Hybrid Class

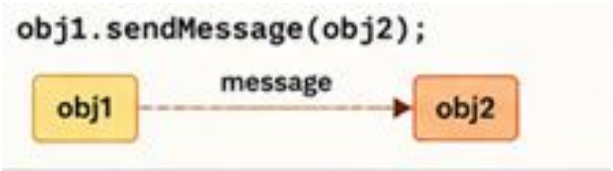
Polymorphism

- It is the concept by which a method or object takes different forms in different. environments.
  - E.g., the operator + adds two whole numbers, two decimal point numbers, strings.
    - Symbol * (star) shows polymorphic behavior.
    - It is used to multiply two values, incorporate comments such as /* and */, import a package into a program like

      import java.lang.*;.

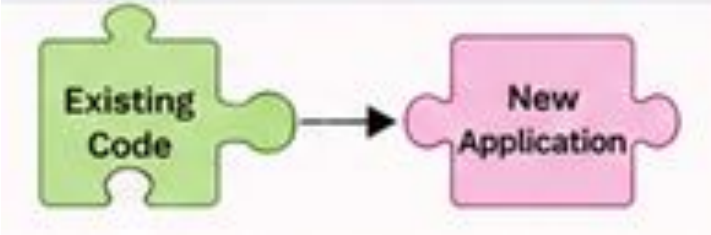
- Message Passing
  - Message passing means one object requests another object to perform an action by calling its method.





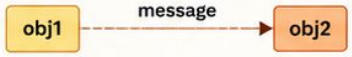
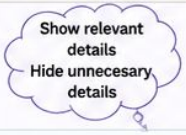
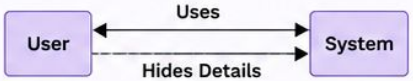
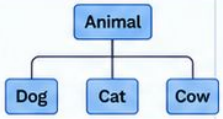
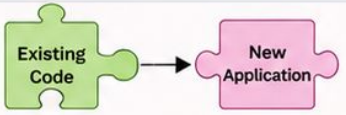
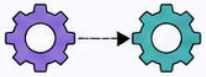
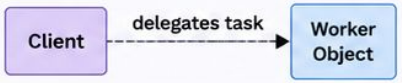
- Reusability
  - Code written once can be reused in many places. Inheritance and classes improve reusability.
  - Example: A Vehicle class can be reused by Car, Bike, and Bus classes.





- **Delegation**
  - Delegation means an object hands over a task to another object instead of doing it itself.
  - Computer delegates printing work to Printer.

Characteristics of Object-Oriented Programming (OOP)

Characteristic	Description	Illustration / Example
1 Class 	A class is a blueprint or template that defines the data (attributes) and behavior (methods) that objects of the class will have.	`class Car { String brand; String color; void start(){ ... } void stop(){ ... } }`
2 Object 	An object is an instance of a class. It represents a real-world entity and has state (data) and behavior (methods).	`Car myCar = new Car();`  myCar
3 Methods and Message Passing 	Objects communicate with each other by sending messages. A message is a request to invoke a method.	`obj1.sendMessage(obj2);` 
4 Data Abstraction 	Data abstraction means showing only essential information to the user and hiding implementation details.	
5 Encapsulation and Data Hiding 	Binding data (attributes) and methods that operate on the data into a single unit (class) and restricting direct access to some of the object's components.	`class Account { private double balance; public void deposit(double amt){...} public double getBalance() {... } }`
6 Inheritance 	A mechanism in which a new class (acquired class) inherits properties and behaviors from an existing class (base class).	`class Dog extends Animal { void bark() { ... } }` 
7 Reusability 	OOP promotes reusability by allowing existing classes, methods, and components to be used in new programs.	
8 Polymorphism  Same operation, Different forms	The ability of an object to take many forms. The same method name can behave differently for different objects.	`shape.draw();` 
9 Delegation 	An object delegates a task to another object to do the work on its behalf.	

COHESION, COUPLING & REUSABILITY IN OOP

1. COHESION (Within a Class)

Cohesion is the degree to which the elements inside a class belong together.
High cohesion is desirable.

HIGH COHESION (Good) ✓

Student

Data Members
id
name
marks
Methods
read()
display()
calculateGrade()

All data and methods are related to Student.



LOW COHESION (Bad) ✗

Student

Data Members
id
name
Methods
read()
display()
playMusic()
calculateSalary()
sendEmail()

Methods are unrelated to Student.



2. COUPLING (Between Classes)

Coupling is the degree of inter-dependence between the pairs of classes.
Low coupling is desirable.

LOW COUPLING (Good) ✓

Student

Data Members
id
name
marks
Methods
display()
printReport()

uses →

Printer

Methods
print()

Student class uses Printer class but both classes have their own responsibilities.

✓ LOW COUPLING

HIGH COUPLING (Bad) ✗

Student

Methods
read(), display(), print(), scan(), connectInternet(), playMusic(), sendEmail(), calculateSalary()

Too many responsibilities and dependencies in one class.

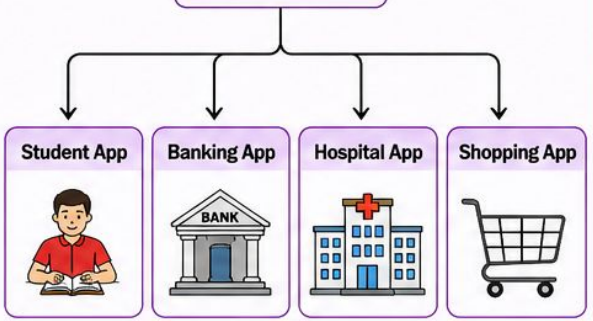
✗ HIGH COUPLING

3. REUSABILITY

Reusability is the ability to reuse the previously defined classes and objects.

Calculator

Methods
add()
sub()
mul()
div()



The same class (Calculator) is reused in many applications.

✓ REUSABILITY

KEY POINTS

- **Cohesion** → Within a class (Keep related things together)
- **Coupling** → Between classes (Keep dependency low)
- **Reusability** → Use the same class again and again

THE GOLDEN RULE

HIGH COHESION + LOW COUPLING = GOOD DESIGN

BENEFITS

- Easy Maintenance
- Less Error
- Better Understandability
- Extensibility
- Reusability



C vs JAVA – 6 Key Differences

#	Aspect	C	JAVA
1	Approach	**Procedure-Oriented** Focuses on functions/procedures. Program is divided into functions.	**Object-Oriented** Focuses on objects and classes. Program is divided into objects.
2	Language Level	**Middle-Level Language** Provides a mix of high-level and low-level features.	**High-Level Language** Closer to human language and easy to understand and maintain.
3	Platform Dependency	**Platform-Dependent** Executable works only on the specific OS for which it is compiled.	**Platform-Independent** Compiled byte code runs on any OS using JVM (Write Once, Run Anywhere).
4	Memory Management	**Manual Memory Management** Programmer allocates and deallocates memory using malloc() and free().	**Automatic Memory Management** Memory is managed automatically by Garbage Collector.
5	Pointers	**Supports Pointers** Allows direct access to memory and pointer arithmetic.	**Does Not Support Pointers** Does not allow direct memory access (no pointer arithmetic).
6	Inheritance	**Does Not Support Inheritance** C is not an object-oriented language. No support for inheritance.	**Supports Inheritance** Java is a pure object-oriented language. Supports inheritance through classes.

Summary

C is suitable for system programming, hardware related tasks and performance-critical applications.

Java is suitable for large-scale, secure, platform-independent and enterprise applications.

Differences between C and JAVA

Parameter	C	JAVA
Language Level	Middle-level language	High-level language
Programming Paradigm	Structural, procedural	Object-oriented
Approach	Top-down	Bottom-up
Compilation	Compiled language	Compiled and Interpreted language
Platform Dependency	Platform dependent	Platform-independent (uses Java Virtual Machine)
Memory Management	Manual (functions like malloc, calloc, free)	Automatic (garbage collection)
Threading	Does not support threading	Supports multi-threading
Pointers	Supports pointers	Does not support pointers

Inheritance	Does not support inheritance	Supports inheritance
Error Handling	No built-in exception handling	Supports exception handling
File Extensions	.c	.java
Security	Less secure	More secure (due to lack of pointers, etc.)
Robustness	Less robust	More robust (strong memory management)
Supports	Applet concept is not there	Applets are there
Applications	System applications, embedded systems	Enterprise applications, web applications
Popularity	Less popular compared to Java	More Popular
Object-Oriented Programming (OOP) Features	Lacks OOP concepts like polymorphism, encapsulation	Fully supports OOP concepts

Diff between C++ and JAVA

Comparison Index	C++	Java
Platform-independent	C++ is platform-dependent.	Java is platform-independent.
Mainly used for	C++ is mainly used for system programming.	Java is mainly used for application programming. It is widely used in Windows-based, web-based, enterprise, and mobile applications.
Goto	C++ supports the goto statement.	Java doesn't support the goto statement.
Multiple inheritance	C++ supports multiple inheritance.	Java doesn't support multiple inheritance through class. It can be achieved by using interfaces in java.
Operator Overloading	C++ supports operator overloading.	Java doesn't support operator overloading.
Object-oriented	Partially	Fully

Pointers	C++ supports pointers. You can write a pointer program in C++.	Java supports pointer internally. However, you can't write the pointer program in java. It means java has restricted pointer support in java.
Compiler and Interpreter	C++ uses compiler only. C++ is compiled and run using the compiler which converts source code into machine code so, C++ is platform dependent.	Java uses both compiler and interpreter.
Call by Value and Call by reference	C++ supports both call by value and call by reference.	Java supports call by value only. There is no call by reference in java.
Structure and Union	C++ supports structures and unions.	Java doesn't support structures and unions.
Thread Support	C++ doesn't have built-in support for threads. It relies on third-party libraries for thread support.	Java has built-in thread support.

Tokens in Java Programs

- ***A Token is a individual/smallest unit of a program element which is recognized by the compiler.***
- ***A program contains set of tokens such as***
  - ***Identifiers or names*** This is meant for variables, methods, classes, etc.
  - ***Keywords*** These are special words defined in Java and represent a set of instructions.
  - ***Literals*** These are values represented by a set of characters, digits, etc.
  - ***Separators*** These include comma, semicolon, period (.), (), [], etc.
  - ***Operators -*** Operators are mostly represented by symbols such as + , −,*, etc.

Identifiers or Names

- Anything stored in the computer memory has a name or identifier which identifies that section of the memory
- Rules:

1. It may be a single word or a combination of words with no space between words.
2. It should not be a keyword of Java.
3. It should not be Boolean literal, that is, true or false.
4. It should not be **null** literal.
5. It should not start with a digit but it can start with an underscore.
6. It can comprise one or more unicode characters as well as digits.

Conventions for Writing Names

- Packages - Names of packages are completely in lower-case letters such as mypackage, java.lang.
- For Class - Names of classes and interfaces start with an upper-case letter.
- For Methods - Names of methods start with a lower-case character.
- For variables - Names of variables should start with a lower-case character.

Keywords in Java

- The keywords represent groups of instructions for the compiler.
- These are special tokens that have a predefined meaning and their use is restricted.
- keywords cannot be used as names of variables, methods, classes, or packages.
- These are written in the lower case.

abstract	continue	for	new	switch
assert	default	goto	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

JAVA KEYWORDS

Keywords are reserved words in Java that have a special meaning and cannot be used as identifiers (class, variable, method or object names).

53 KEYWORDS IN JAVA

1 DATA TYPES

byte short int
long float double
char boolean void

Used to declare variables and specify data types.

2 ACCESS MODIFIERS

public protected
private default

Used to control visibility and access to classes, methods and variables.

3 CONTROL FLOW

if for
else while
switch do return
case break
default continue

Used to control the flow of execution in a program.

4 CLASS & OBJECT

class new
interface this
extends super
implements

Used in the definition of classes, inheritance and object creation.

5 EXCEPTION HANDLING

try
catch throws
finally
throw

Used to handle exceptions and error management.

6 MODIFIERS

abstract transient
final volatile
static native
strictfp
synchronized

Used to modify classes, methods and variables.

7 PACKAGE RELATED

package instanceof
import

Used in package declaration, importing and type checking.

8 BOOLEAN LITERALS

true false

Used as boolean constant values.

9 LITERALS

null

Used to represent no reference.

10 RESERVED (FUTURE USE)

goto enum assert
const

Reserved for future use. (enum is used from Java 5 onwards.)

NOTE
These keywords cannot be used as identifiers.

TOTAL KEYWORDS: 53

Remember:
- Java is case-sensitive.
- Keywords must be written in lowercase.
- They have predefined meaning in Java.

```
class Example {
    public static void main(String[] args) {
        int value = 10;
        if (value > 0) {
            System.out.println("Positive");
        }
    }
}
```

Keywords make Java programs powerful!

Literals in Java

- A literal is a constant value
- The value may be a number (either whole or decimal point number) or a sequence of characters which is called String literal, Boolean type, etc.
- Types of Literals:

  1. Integer literals - 6, 453, 34789
  2. Floating point literal - 3.14159, 567.78
  3. Boolean literal - two values—true or false
  4. Character literal - Characters are represented in single quotes such as ‘A’, ‘H’, ‘k’,
  5. String literal - “AP”, “Amaravati”, “Vijayawada”,
  6. Null literal - null

Separators

In Java, **Separators** (also called **punctuators**) are special symbols used to separate program elements such as statements, blocks, method parameters, and array indices. According to the Java Language Specification, Java has **12 separators**.

Separator	Name	Purpose	Example
( )	Parentheses	Enclose method parameters and expressions	sum(a, b)
{ }	Braces	Define a block of code or class body	{ int x = 10; }
[ ]	Brackets	Declare and access arrays	int[] a; a[0]
;	Semicolon	Terminates a statement	int x = 10;

,	Comma	Separates variables, parameters, or arguments	int a, b, c;
.	Dot (Period)	Access members of a class/object	System.out.println()
...	Ellipsis	Variable-length arguments (varargs)	void show(int... n)
@	At Symbol	Introduces annotations	@Override
::	Method Reference Operator	References a method	System.out::println

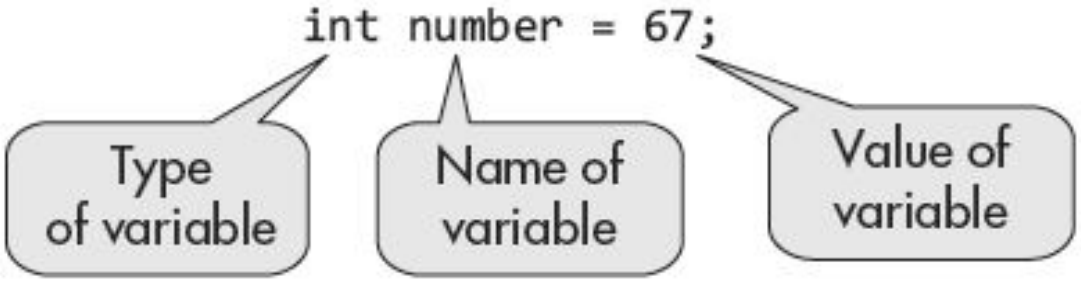
```
class Demo {
    public static void main(String[] args) {
        int[] a = {10, 20, 30};

        for (int i = 0; i < a.length; i++) {
            System.out.println(a[i]);
        }
    }
}
```

Separator	Usage
{ }	Class, method, and loop blocks
( )	Method declaration, main(), for(), println()
[ ]	Array declaration and indexing
;	Ends statements
.	Accesses System.out.println
,	Separates values in array

Declaration of Variables

- Variables are objects which contains some values, those values may change in the program.
- A variable is declared by first writing its type, followed by its name or identifier.



JAVA STATEMENTS

A Java program consists of a set of instructions.
These instructions are called **statements**.

TYPES OF JAVA STATEMENTS

1. EMPTY STATEMENT

An empty statement contains only a semicolon (;).

It performs no action.

2. VARIABLE DECLARATION STATEMENT

Used to declare variables of different data types to store values.

3. LABELED STATEMENT

A statement preceded by an identifier and a colon (:).

Used with break and continue in loops.

4. EXPRESSION STATEMENT

Performs an operation.

Examples: assignment, method call, increment, object creation, etc.

5. CONTROL STATEMENTS

A. SELECTION STATEMENTS

Used for decision making.

Types:

- if
- if-else
- else-if ladder
- switch

B. ITERATION STATEMENTS

Used to repeat a block of code.

Types:

- while
- do-while
- for
- enhanced for (for-each)

C. JUMP STATEMENTS

Transfer control from one part to another.

Types:

- break
- continue
- return
- throw

6. SYNCHRONIZED STATEMENT

Used in multithreading to control access to shared resources.

Ensures that only one thread executes a block at a time.

7. GUARDING STATEMENTS (EXCEPTION HANDLING)

Used to handle runtime errors (exceptions) and maintain normal flow of the program.

Types:

- try
- catch
- finally
- throw / throws

JAVA STATEMENTS – TYPES AND CODE SNIPPETS

TYPES OF JAVA STATEMENTS

CODE SNIPPETS (For Understanding)

1 EMPTY STATEMENT



```
for(int i = 0; i < 5; i++);    // Empty statement ( ; ) acts as the loop body
```

2 VARIABLE DECLARATION STATEMENT



```
int age = 20;
double salary = 45000.50;
char grade = 'A';
```

3 LABELED STATEMENT

label:

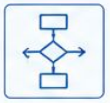
```
outer:
  for(int i = 1; i <= 3; i++) {
      for(int j = 1; j <= 3; j++) {
          if(i == 2)
              break outer;   // exit outer loop
      }
  }
```

4 EXPRESSION STATEMENT



```
x = 10;          // assignment      x++;          // increment
--x;             // decrement       display();    // method call
new Student();   // object creation
```

5 CONTROL STATEMENTS



A) SELECTION STATEMENTS

```
if(age >= 18)
   System.out.println("Eligible");
else
   System.out.println("Not Eligible");
```

B) ITERATION STATEMENTS

```
for(int i = 1; i <= 5; i++)
   System.out.println(i);

while(i < 5) {
   i++;
}
```

C) JUMP STATEMENTS

```
break;          // terminates loop/switch
continue;       // skips to next iteration
return;         // returns from method
throw new Exception(); // throws exception
```

6 SYNCHRONIZED STATEMENT



```
synchronized(this) {          // only one thread
    count++;                  // can execute this
}                             // block at a time
```

7 GUARDING STATEMENTS (EXCEPTION HANDLING)



```
try {
    int x = 10 / 0;
} catch(ArithmeticException e) {
    System.out.println(e);
} finally {
    System.out.println("Done");
}
```

// try: risky code
// catch: handles exception
// finally: always executed



Note: Every Java program is built using one or more of the above statements.

These statements perform different actions and together control the flow of execution in a program.

Java Statements

- A Java program consists of a set of instructions, these instructions are the statements in Java.
- Empty statement –
  - An empty statement in Java is written as a single semicolon.
  - For example, you can use it to indicate an empty loop body of a for loop:

```
for(int i = 0; i < 10; a[i++]++) // Increment array elements
    /* empty */; // Loop body is empty statement
```

- Variable declaration statements – a variable to store value

- Labelled statements

  A labeled statement is simply a statement that has been given a name by prepending a identifier and a colon to it.

  Labels are used by the break and continue statements.

```
rowLoop: for(int r = 0; r < rows.length; r++)
{
        // A labeled loop
                colLoop: for(int c = 0; c < columns.length; c++)
        {
                        // Another one break rowLoop;
                        // Use a label
                }
}
```

- Expression statements:

  var=expr; expr++; ++expr; method(); newType(); --expr; expr- -;

- Control statements
  - Selection statements
  - Iteration statements
  - Jump statements - break, continue, return and throw
- Synchronized statement
- Guarding statement – Exceptions - try, catch block and finally

Command line arguments

- A Java application can accept any number of arguments from the command line.
- These arguments can be passed at the time of running the program.
- length property array is used to find number of arguments

User input to programs

- There are various ways to read input from the keyboard, the java.util.Scanner class is one of them.
- The **Java Scanner** class breaks the input into tokens using a delimiter that is whitespace by default.
- It provides many methods to read and parse various primitive values.
- Part of util package.
- Programs discussed in text book:
  - Performing arithmetic operations on two numbers
  - Finding Area of a circle

Method	Description
public String next()	it returns the next token from the scanner.
public String nextLine()	it reads the given input as one line irrespective of white space and returns the value as a string.
public byte nextByte()	it scans the next token as a byte.
public short nextShort()	it scans the next token as a short value.
public int nextInt()	it scans the next token as an int value.
public long nextLong()	it scans the next token as a long value.
public float nextFloat()	it scans the next token as a float value.
public double nextDouble()	it scans the next token as a double value.

ESCAPE SEQUENCES IN JAVA

Escape sequences are special characters that begin with a backslash (\).
They are used in strings and character literals for formatting and special characters.

ESCAPE SEQUENCE	DESCRIPTION	EXAMPLE	OUTPUT
\n	New line	"Hello\nJava"	Hello Java
\t	Horizontal tab	"Java\tProgramming"	Java Programming
\b	Backspace (deletes the previous character)	"ABC\bD"	ABD
\r	Carriage return (moves cursor to the beginning of line)	"Hello\rJava"	Java *(overwrites the beginning)*
\f	Form feed (page break)	"Hello\fWorld"	Hello --- World *(form feed character)*
\'	Single quotation mark	'\''	'
\"	Double quotation mark	"He said, \"Hi\""	He said, "Hi"
\\	Backslash	"C:\\Java"	C:\Java

UNICODE Escape sequences

- \uXXXX represents a **Unicode escape sequence** in Java, where XXXX is a **4-digit hexadecimal Unicode value**.
- Unicode escape sequences enable Java programs to display international currency symbols, mathematical symbols, and characters from almost every language.
- The **Indian Rupee symbol (₹)** is represented by **\u20B9** and is supported in Java 7 and later.

UNICODE Escape Characters

Char	Unicode	Escape Sequence	Description
&	U+0026	\u0026	Ampersand
•	U+2022	\u2022	Bullet
	U+25E6	\u25E6	White Bullet
·	U+2219	\u2219	Bullet Operator
‣	U+2023	\u2023	Triangular Bullet
⁃	U+2043	\u2043	Hyphen Bullet
°	U+00B0	\u00B0	Degree
∞	U+221E	\u221E	Infinity

$	U+0024	\u0024	Dollar
€	U+20AC	\u20AC	Euro
£	U+00A3	\u00A3	Pound
¥	U+00A5	\u00A5	Yen / Yuan
¢	U+00A2	\u00A2	Cent
₹	U+20B9	\u20B9	Indian Rupee
Rs	U+20A8	\u20A8	Rupee
₱	U+20B1	\u20B1	Peso
₩	U+20A9	\u20A9	Korean Won
฿	U+0E3F	\u0E3F	Thai Baht
₫	U+20AB	\u20AB	Dong
₪	U+20AA	\u20AA	Shekel

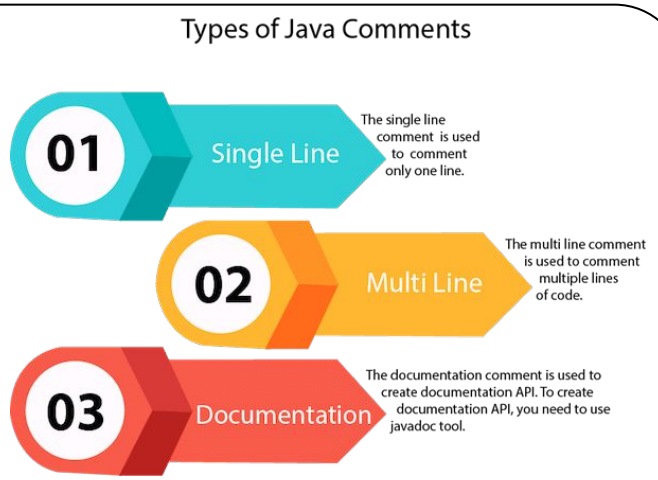
Intellectual Property Codes			
©	U+00A9	\u00A9	Copyright
®	U+00AE	\u00AE	Registered Trademark
℗	U+2117	\u2117	Sound Recording Copyright
™	U+2122	\u2122	Trademark
℠	U+2120	\u2120	Service Mark

Octal Escape Sequence in Java

- An **Octal Escape Sequence** represents a character using its **octal (base-8)** ASCII value.
- It begins with a **backslash (\)** followed by **1 to 3 octal digits i.e., 0-7**.

Decimal	Character	Java Escape – Octal Code
65	A	\101
66	B	\102
67	C	\103
90	Z	\132
97	a	\141
98	b	\142
99	c	\143
122	z	\172
48	0	\060
49	1	\061
50	2	\062
32	Space	\040

Comments



- 3 types
- Single Line Comment
  - The single-line comment is used to comment only one line of the code.
  - // Not executed
- Multiline Comment -/* and */
- Documentation comments are usually used to write large programs for a project or software application as it helps to create documentation API. These APIs are needed for reference, i.e., which classes, methods, arguments, etc., are used in the code.
  - To create documentation API, we need to use the **javadoc tool**. The documentation comments are placed between /** and */.

Doc Comments

```
F:\Courses\JAVA26-27\UNIT 1 PPT and Material\Programs>javac DocumentationComment.java

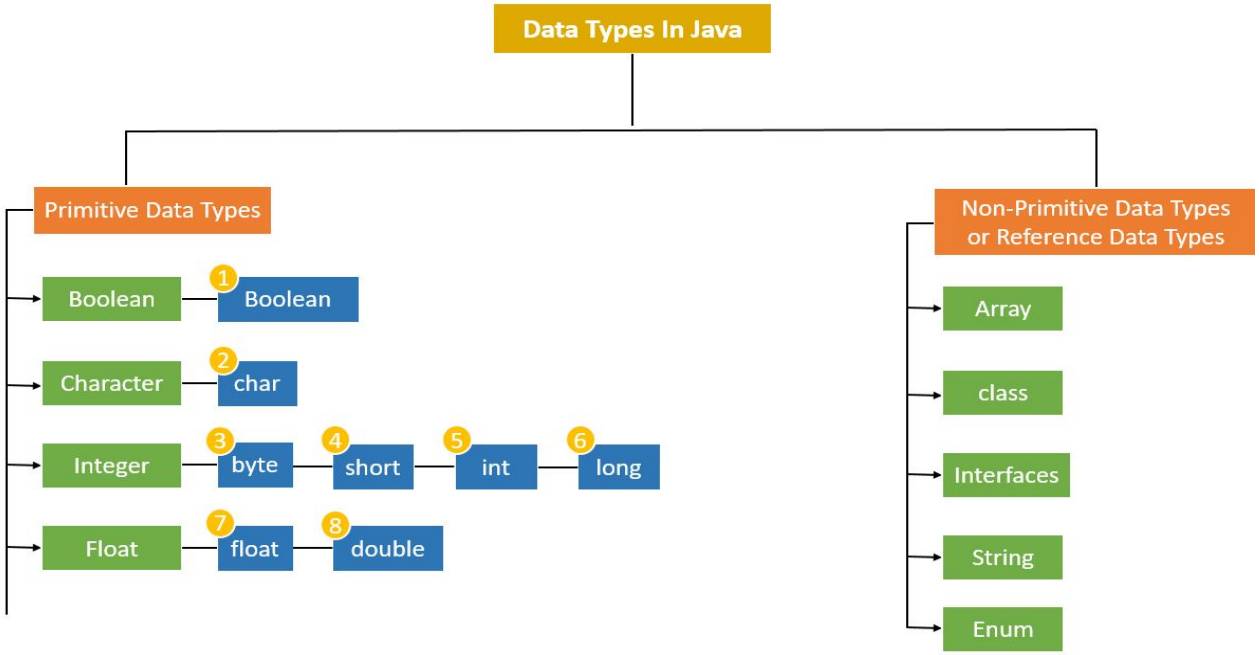
F:\Courses\JAVA26-27\UNIT 1 PPT and Material\Programs>java DocumentationComment
Addition of numbers: 60

F:\Courses\JAVA26-27\UNIT 1 PPT and Material\Programs>javadoc DocumentationComment.java
Loading source file DocumentationComment.java...
Constructing Javadoc information...
```

Data Types in Java

- Java defines eight primitive (basic) types of data
- 1. Integral types—**byte, short, int, long**
- 2. Floating point types—**float, double**
- 3. Character type—**char**
- 4. Boolean type—**boolean**

- There is a non-data type called **void**.
- This type is used for methods that do not return any value.



Declaration of Variables

- A variable is declared by first declaring its *type* followed by its identifier or name.
- The name or identifier should not be a keyword of Java language.
- It can be of any length and may consist of characters and digits.



Non-primitive Types

- These are the class and interface types. The name of a class or interface is the name of type. A class object is declared as

```
Class_identifier object_identifier;
```

Integers

- Integers are whole numbers, that is, they represent numbers that do not have a fractional part.
- In Java, these numbers may be positive or negative.

Table 3.1 Integer types and ranges

Type name	*Memory allocated in bytes*	*Range of values of variables*
byte	1 byte (8 bits)	–128 to 127
short	2 bytes (16 bits)	–32,768 to 32,767
int	4 bytes (32 bits)	–2,147,483,648 to 2,147,483,647
long	8 bytes (64 bits)	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807

Characters

- Java supports Unicode for the representation of characters.
- Unicode supports all the character sets of all the major languages of the world.
- The initial version of Unicode allocated 2 bytes for storing characters.
- The range of values for characters in the initial version of Unicode comprised from ‘\u0000’ to ‘\uffff ’ that is from 0 to 65535 both end values inclusive.
  - ‘A’ - \u0041 – 65
  - ‘1’ - \u0031 - 49

Floating-point numbers

- The numbers that are not whole numbers, or those that have fractional part, Examples are 3.141, 476.6, and so on.
- Java supports two types of such numbers.
- **Float:** This type is used for single-precision decimal floating point numbers, that is, 7(or 8) digits after the decimal point.
- These numbers are stored in 4 bytes .
- **Double** This type is used for double-precision decimal floating point numbers, that is, 15 digits after the decimal point.

The letter f or F is appended to float value otherwise number will be considered as double value

Table 3.3 Range of values of `float` and `double` type variables

Data Type	*Memory allocated*	*Range of values*
`float`	4 byte	1.4e – 45 to 3.4e + 38
`double`	8 byte	4.9e – 324 to 1.8e + 308

1.4E-45 is **scientific notation (exponential notation)**.

It means: 1.4×10^{-45}

E means **"× 10 raised to the power of"**.
E+ means move the decimal point to the **right**.
E- means move the decimal point to the **left**.

POSITIVE EXPONENT (+)

Move the decimal point to the RIGHT.

Expression	Meaning	How to Move	Result
2.5E3	2.5×10^{3}	Move RIGHT 3 places	2.5 → 25.0 → 250. → 2500. = 2500
2.5E2	2.5×10^{2}	Move RIGHT 2 places	2.5 → 25.0 → 250. = 250
2.5E1	2.5×10^{1}	Move RIGHT 1 place	2.5 → 25.0 = 25
2.5E0	2.5×10^{0}	No Movement	2.5 = 2.5

NEGATIVE EXPONENT (–)

Move the decimal point to the LEFT.

Expression	Meaning	How to Move	Result
2.5E-1	2.5×10^{-1}	Move LEFT 1 place	2.5 → 0.25 = 0.25
2.5E-2	2.5×10^{-2}	Move LEFT 2 places	2.5 → 0.25 → 0.025 = 0.025
2.5E-3	2.5×10^{-3}	Move LEFT 3 places	2.5 → 0.25 → 0.025 → 0.0025 = 0.0025

boolean Data Type

- For dealing with logical statements, variable of type boolean is supported.
- boolean occupies **1 bit conceptually** (its exact memory representation is JVM-dependent). It stores **only two values**: true or false
  - It is mainly used in: Decision making (if, else)
  - Loops (while, for)
  - Logical operations (&&, ||, !)
- **Unlike C/C++, Java does not allow 1 or 0 to represent true or false. Only the boolean literals true and false are valid.**

Type Casting

- **Type Casting** (or **Type Conversion**) is the process of converting data from **one data type to another data type**.
- It is required when an expression contains **different data types**.

```
int a = 10;
double b = 20.5;
double c = a + b;
```

Here, a is an int and b is a double. Before addition, Java automatically converts a

into double.

- Some type castings are done automatically by a compiler, while others have to be done by the programmer.

- **Types of Type Casting**
- There are **two types** of type casting:
  - **Automatic Type Conversion (Widening)**
    - Done **automatically by the Java compiler**.
    - No explicit casting is required.
  - **Conditions**
    - The source and destination data types must be **compatible**.
    - The destination data type must be **larger** than the source data type.

byte - > Short - > int-> long **float -> double**
char-> int

Casting Incompatible Types

- Explicit Type Casting is the process of converting a larger data type into a smaller compatible data type.
- It is performed manually by the programmer using the cast operator.
- Explicit casting is also called Narrowing Conversion because the destination data type has a smaller range than the source data type.
- Syntax

  variable = (target_type) value;

  double->float>long>int->short>byte

Tasks

- Print area of circle and area of rectangle using double data types.

Scope of variable

- Variable to be declared within any block
- A block is begin with an opening curly brace and ended by a closing curly brace
- A block defines a *scope*.
- Thus, each time you start a new block, you are creating a new scope. A scope determines what objects are visible to other parts of your program.
- Many other computer languages define two general categories of scopes: **global and local.**
- As a general rule, variables declared inside a scope are not visible (that is, accessible) to code that is defined outside that scope.
- Scopes can be nested. For example, each time you create a block of code, you are creating a new, nested scope.

Literal Constants

- A literal is a constant value written directly in the source code.

Type	Example	Description
Integer Literal	10, -25, 1000	Whole numbers
Floating-point Literal	3.14, 2.5f, 45.67	Decimal numbers
Character Literal	'A', '9', '#'	Single character enclosed in single quotes
String Literal	"Hello", "Java"	Sequence of characters enclosed in double quotes
Boolean Literal	true, false	Logical values
Null Literal	null	Represents no object/reference

Symbolic Constants

- A symbolic constant is a variable whose value does not change throughout the program.
- A symbolic constant is declared using the **final** keyword.
- Once initialized, **its value cannot be changed**.
- By convention, symbolic constant names are written in **UPPER_CASE**.
  - Examples: PI, MAX_SIZE, MIN_AGE, INTEREST_RATE.

Types of variables

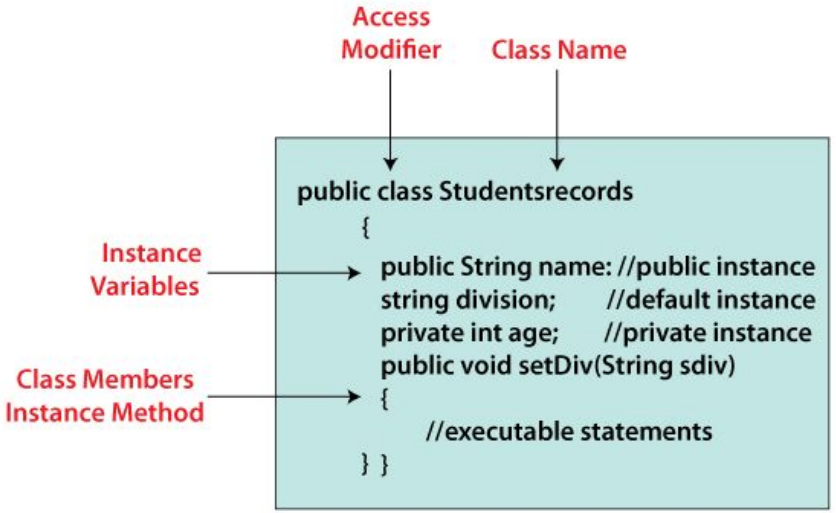
- **Local variables** – inside a method or parameters of a method. **Local variables must be initialized before use**, as they don't have a default value and the compiler **won't allows us to use** an uninitialized value.

In other words, local variables don't have default values.

- **Instance variables** – Declared anywhere in a class but outside of the method. Each object has its own copy of these variables.
- **Static variable** – declaration of t

static. Only one copy is maintain

Write the differences between local, instance and static variables in JAVA.



TYPES OF VARIABLES IN JAVA

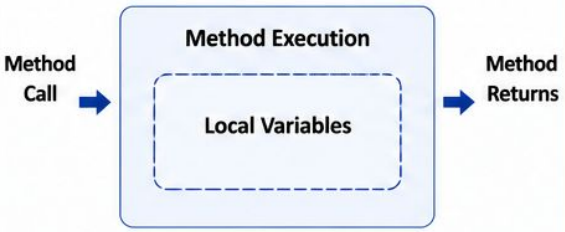
1. LOCAL VARIABLES

- Declared **inside a method** or **parameters of a method**.
- Local variables must be **initialized before use**.
- Don't have **default value**.
- Compiler won't allows us to use an **uninitialized value**.
- Exist only **while the method executes**.

BELONGS TO

Method

MEMORY DIAGRAM



★ Created when the method is called and destroyed when the method returns.

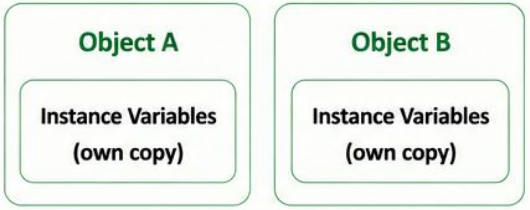
2. INSTANCE VARIABLES

- Declared anywhere in a class but **outside of the method**.
- Each object has its **own copy** of these variables.
- Have **default values**.
- Exist as long as the **object exists**.

BELONGS TO

Object

MEMORY DIAGRAM



★ Every object has its own copy in heap memory.

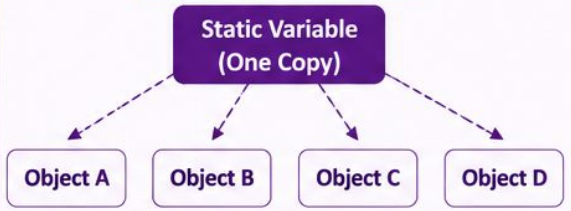
3. STATIC VARIABLES

- Declaration of these variables precede with a keyword **static.**
- Only **one copy** is maintained.
- Shared by all objects of the class.
- Can be accessed using **ClassName.variableName.**
- Exist as long as the **class is loaded** in memory.

BELONGS TO

Class

MEMORY DIAGRAM



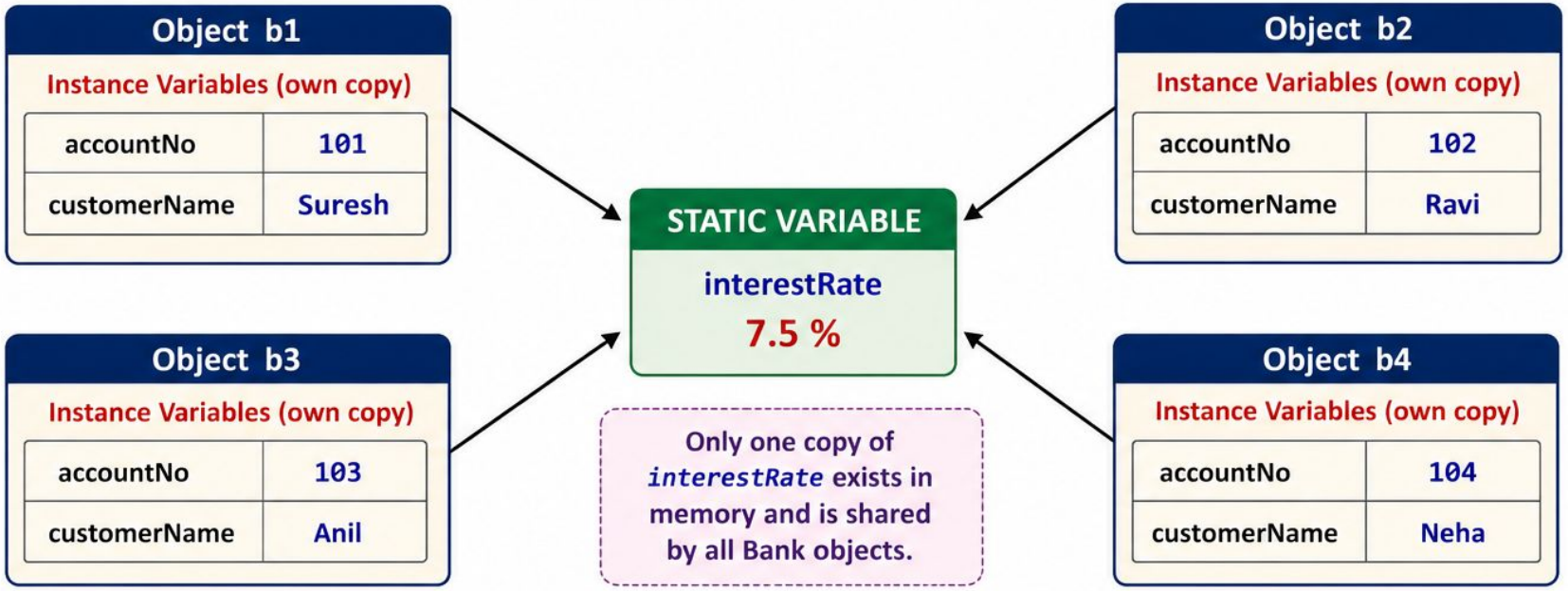
★ Only one copy in method area and shared by all objects.

Key Point: Local variables → Method specific, Instance variables → Object specific, Static variables → Class specific.

Feature	Local Variable	Instance Variable	Static Variable
Declared	Inside a method	Inside a class	Inside a class using static
Belongs to	Method	Object	Class
Default Value	❌ No	✅ Yes	✅ Yes
Memory Created	When method is called	When object is created	When class is loaded
Access	Directly within the method	Using an object	Using class name or directly in the same class

STATIC VARIABLE – Bank Interest Rate Concept

All Bank objects share one static variable (*interestRate*).
Each object has its own copy of instance variables (*accountNo*, *customerName*).



Creating Objects

```
Bank b1 = new Bank(101, "Suresh");
Bank b2 = new Bank(102, "Ravi");
Bank b3 = new Bank(103, "Anil");
Bank b4 = new Bank(104, "Neha");
```

Changing Static Variable

```
Bank.interestRate = 8.0;  // change once
```

Change is reflected in all objects because they share the same static variable.

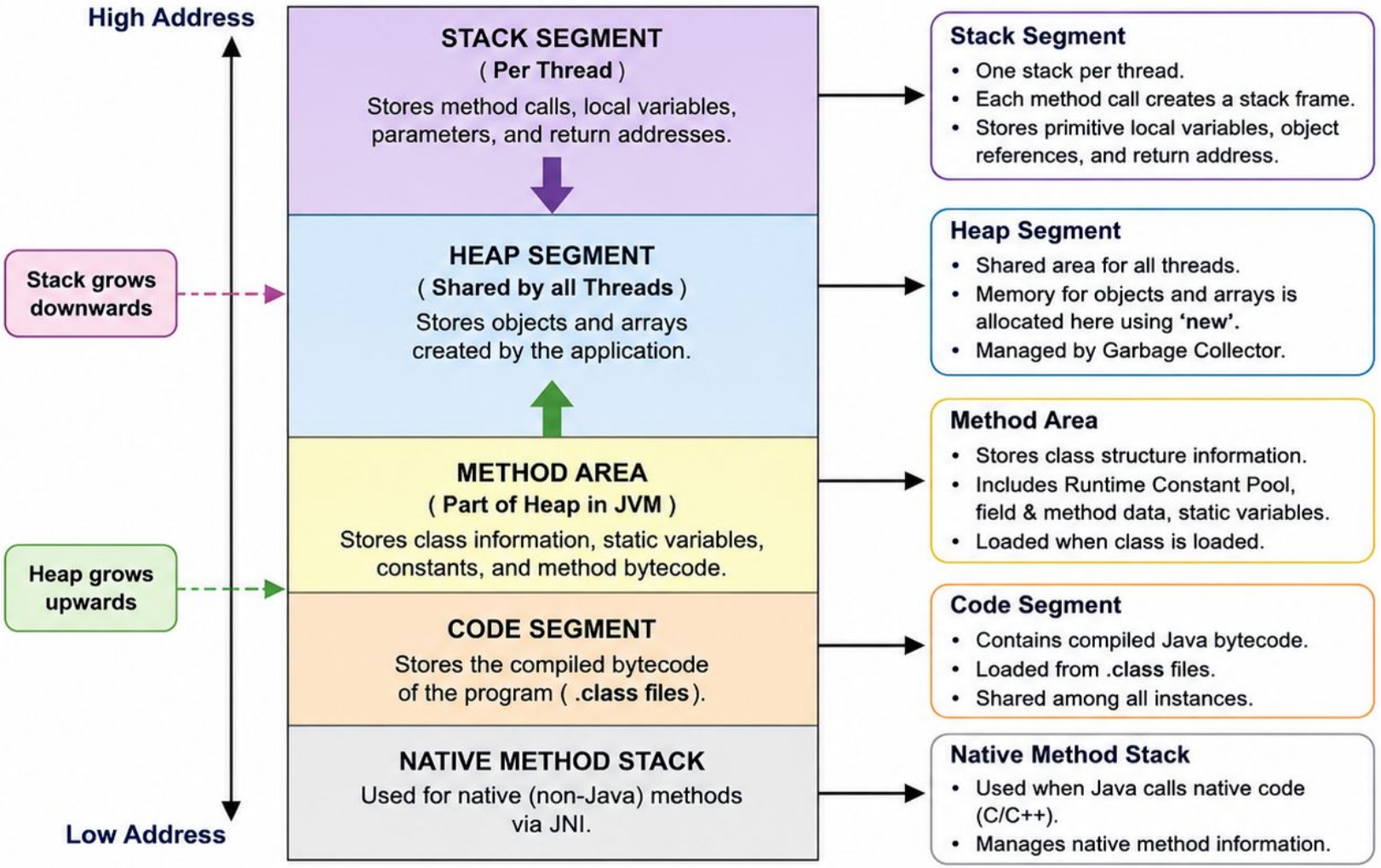
Output After Change

```
101   Suresh   8.0%
102   Ravi     8.0%
103   Anil     8.0%
104   Neha     8.0%
```

Key Point: Instance variables are separate for each object, but static variables are common (one copy) for all objects.

Memory Allocation to a Java Process

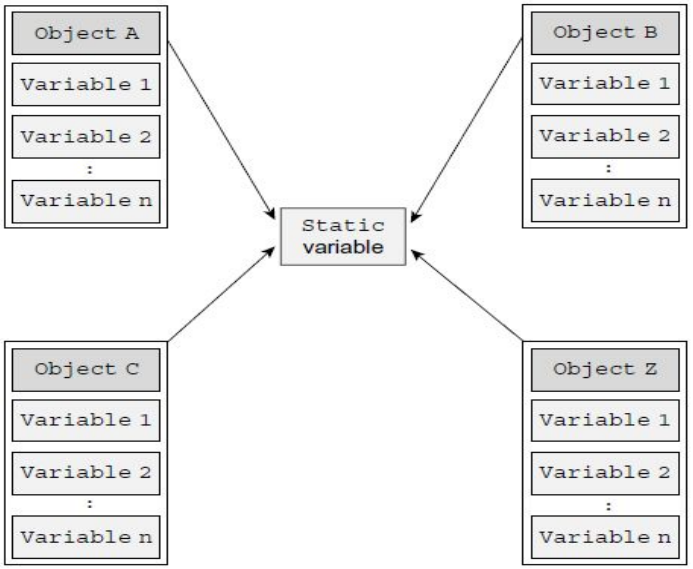
When a Java program runs, the JVM allocates memory to the process in the following regions.



Note : Actual memory layout may differ across JVM implementations, but conceptually a Java process uses these memory regions.

Static Variable

- The variables in a class may be modified by modifier static.
- The non-static variables declared in a class are instance variables.
- Each object of the class keeps a copy of the values of these variables.
- The static variables are class variables.
- The static variables are accessed through class reference, whereas the instance variables are accessed through class object reference.



Attribute Final

- The value of a variable declared final cannot be changed in the program.
- It makes the variable a constant and must be initialized at the time of declaration.
- A few examples of declarations are as:

```
final double PI = 3.14159;
final int M = 900;
final double X = 7.5643;
```



Operators

- **Java operators are symbols used to perform operations on variables and values.**

- Arithmetic operators
- Compound assignment operators
- Increment and decrement operators
- Relational operators
- Boolean logical operators
- Bitwise logical operators
- Shift operators
- Special operators
- If an operator takes only one operand for its operation, it is called unary operator.
- If it takes two operands, it is called binary operator.
- If it takes three operands, it is called ternary operator.

ARITHMETIC OPERATORS IN JAVA

Arithmetic operators are used to perform mathematical operations on numeric values.

Operator	Name	Description	Example	Explanation	Result
+	Addition	Adds two operands	`int a = 10, b = 5;` `int c = a + b;`	c = 10 + 5	15
−	Subtraction	Subtracts right operand from left operand	`int a = 10, b = 5;` `int c = a - b;`	c = 10 - 5	5
*	Multiplication	Multiplies two operands	`int a = 10, b = 5;` `int c = a * b;`	c = 10 * 5	50
/	Division	Divides left operand by right operand	`int a = 10, b = 5;` `int c = a / b;`	c = 10 / 5	2
%	Modulus (Remainder)	Returns remainder after division	`int a = 10, b = 3;` `int c = a % b;`	c = 10 % 3	1

++	Increment	Increases value by 1	`int a = 5;`
			`int b = ++a;`
			`// or a = a + 1;`
				a = 6, b = 6	6

−−	Decrement	Decreases value by 1	`int a = 5;`
			`int b = --a;`
			`// or a = a - 1;`
				a = 4, b = 4	4

Note: In Java, if both operands are integers, division (/) returns integer result (decimal part is discarded). To get decimal result, at least one operand must be float or double.

Increment (++) and Decrement (––) Operators

Table 3.9 Increment and decrement operators

Operator	*Symbol*	*Code*	*Description*
Pre-increment	++	++x	First increases x by unity then evaluates expressions
Post-increment	++	x++	First evaluates expression with current value of x, and then, increases x by unity
Pre-decrement	--	--x	First decreases x by unity then evaluates expression
Post-decrement	--	x--	Evaluates the expression with current value of x, and then decreases x by unity

```
int x=20, y=10,z;
z=++x + y;
```

z? and x?

```
z=x++ + y;  if x=20, y=10
```

RELATIONAL OPERATORS IN JAVA

Relational operators are used to compare two values. They return boolean result: **true** or **false**.

Operator	Name	Description	Example	Explanation	Result

==	Equal to	Returns true if both operands are equal.	`int a = 10, b = 10;`
			`boolean c = (a == b);`
			`System.out.println(c);`
				a is equal to b
				(10 == 10)
					true

!=	Not equal to	Returns true if both operands are not equal.	`int a = 10, b = 5;`
			`boolean c = (a != b);`
			`System.out.println(c);`
				a is not equal to b
				(10 != 5)
					true

>	Greater than	Returns true if left operand is greater than right operand.	`int a = 10, b = 5;`
			`boolean c = (a > b);`
			`System.out.println(c);`
				a is greater than b
				(10 > 5)
					true

<	Less than	Returns true if left operand is less than right operand.	`int a = 10, b = 5;`
			`boolean c = (a < b);`
			`System.out.println(c);`
				a is less than b
				(10 < 5)
					false

>=	Greater than or equal to	Returns true if left operand is greater than or equal to right operand.	`int a = 10, b = 10;`
			`boolean c = (a >= b);`
			`System.out.println(c);`
				a is greater than or equal to b
				(10 >= 10)
					true

<=	Less than or equal to	Returns true if left operand is less than or equal to right operand.	`int a = 5, b = 10;`
			`boolean c = (a <= b);`
			`System.out.println(c);`
				a is less than or equal to b
				(5 <= 10)
					true



Note: Relational operators always return boolean values: **true** or **false**.

JAVA LOGICAL OPERATORS

Logical operators are used to combine boolean expressions.
They return a boolean value: true or false.

OPERATOR	NAME	BEHAVIOR	EXAMPLE EXPRESSION	RESULT
&&	Short-Circuit AND	Returns true if both operands are true. Skips second if first is false.	(5 > 2) && (3 < 4)	true
||	Short-Circuit OR	Returns true if at least one operand is true. Skips second if first is true.	(2 > 5) || (3 < 4)	true
!	Logical NOT	Unary operator; inverts the boolean value.	!(5 > 2)	false
&	Non-Short-Circuit AND	Returns true if both operands are true. Always evaluates both sides.	isValid & checkNext()	Evaluates both sides
|	Non-Short-Circuit OR	Returns true if at least one operand is true. Always evaluates both sides.	isValid | checkNext()	Evaluates both sides
^	Logical XOR	Returns true if exactly one operand is true (operands must be different).	true ^ false → true	true

Note: && and || are short-circuit operators (more efficient). & and | are non-short-circuit operators (always evaluate both sides).

BITWISE OPERATORS IN JAVA

Bitwise operators perform bit-level operations on integer types (**byte**, **short**, **int**, **long**). They work on the binary representation of numbers.

Operator	Name	Description	Example (int a = 5, b = 3)	Binary Representation	Result

&	Bitwise AND	Returns 1 if both corresponding bits are 1; otherwise 0.	`a = 5, b = 3;`
			`int c = a & b;  // c = 1`
				`a = 5 = 0000 0101`
				`b = 3 = 0000 0011`
				`a & b = 0000 0001 (1)`
					1

|	Bitwise OR	Returns 1 if at least one of the corresponding bits is 1; otherwise 0.	`a = 5, b = 3;`
			`int c = a | b;  // c = 7`
				`a = 5 = 0000 0101`
				`b = 3 = 0000 0011`
				`a | b = 0000 0111 (7)`
					7

^	Bitwise XOR	Returns 1 if the corresponding bits are different; otherwise 0.	`a = 5, b = 3;`
			`int c = a ^ b;  // c = 6`
				`a = 5 = 0000 0101`
				`b = 3 = 0000 0011`
				`a ^ b = 0000 0110 (6)`
					6

~	Bitwise NOT (Complement)	Inverts all the bits (0 becomes 1, 1 becomes 0).	`a = 5;`
			`int c = ~a;  // c = -6`
				`a = 5 = 0000 0000 0000 0000 0000 0101`
				`~a = 1111 1111 1111 1111 1111 1010`
				(In 32-bit signed int, this is -6)
					-6

<<	Left Shift	Shifts bits to the left by specified positions. Rightmost bits filled with 0.	`a = 5;` `int c = a << 1;  // c = 10`	`a = 0000 0101` `a << 1 = 0000 1010 (10)`	10
>>	Right Shift (Signed)	Shifts bits to the right by specified positions. Leftmost bit (sign bit) is extended.	`a = 5;` `int c = a >> 1;  // c = 2`	`a = 0000 0101` `a >> 1 = 0000 0010 (2)`	2
>>>	Unsigned Right Shift	Shifts bits to the right by specified positions. Leftmost bits filled with 0.	`a = 5;` `int c = a >>> 1;  // c = 2`	`a = 0000 0101` `a >>> 1 = 0000 0010 (2)`	2

Note:

- All bitwise operators work only with integral types.
- ~, <<, >>, >>> are unary or binary depending on usage.
- For negative numbers, Java uses 2's complement representation.

Quick Example

```
int a = 5;  // 0000 0101
int b = 3;  // 0000 0011

a & b = 1  (0000 0001)
a | b = 7  (0000 0111)
a ^ b = 6  (0000 0110)

~a    = -6  (1111...1010)
a << 1 = 10  (0000 1010)
a >> 1 = 2   (0000 0010)
a >>> 1 = 2  (0000 0010)
```

JAVA SHIFT OPERATORS: >> vs >>>

Shifts all bits to the right

Understanding Signed Right Shift and Unsigned Right Shift

SIGNED RIGHT SHIFT (>>)

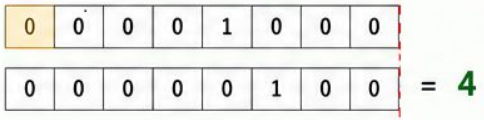
- Copies the sign bit (MSB).
- Preserves the sign of the number.
- Positive remains positive.
- Negative remains negative.

Memory Trick:
>>
= Keep the Sign

Positive Number Example

8 = 00001000

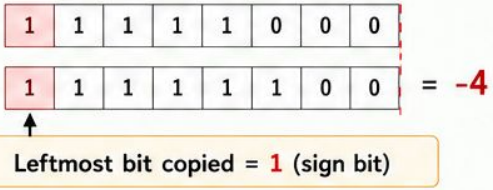
8 >> 1



Negative Number Example

-8 = 11111000

-8 >> 1



UNSIGNED RIGHT SHIFT (>>>)

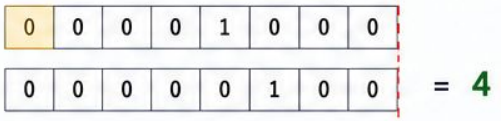
- Always inserts 0 on the left.
- Does NOT preserve the sign.
- Negative numbers become large positive numbers.

Memory Trick:
>>>
= Fill with Zero

Positive Number Example

8 = 00001000

8 >>> 1



Negative Number Example

-8 = 11111000

-8 >>> 1



2'S COMPLEMENT OF 8

2's Complement is a way to represent negative numbers in binary.

Steps: 1) Write the binary number 2) Find 1's complement 3) Add 1

Find 2's Complement of +8 (in 8-bit representation)

STEP 1

Write the binary of +8

0	0	0	0	1	0	0	0
128	64	32	16	8	4	2	1

$+8 = 00001000_2$

STEP 2

Find 1's Complement (invert all bits)

1	1	1	1	0	1	1	1
128	64	32	16	8	4	2	1

1's Complement $= 11110111_2$

STEP 3

Add 1 to the 1's Complement

1	1	1	1	0	1	1	1
							+ 1
1	1	1	1	1	0	0	0

2's Complement of 8 $= 11111000_2$

Verification

8 → 00001000

2's Complement of 8 → 11111000

Add both:

```
  00001000
+ 11111000
-----------
 100000000  (Ignore carry out)
```

Result is 0, which proves it is a correct 2's Complement.

Summary

Positive 8 (8-bit)	:	00001000
1's Complement	:	11110111
Add 1	:	+ 1
2's Complement of 8	:	11111000

ASSIGNMENT OPERATORS IN JAVA



Assignment operators are used to assign a value to a variable.
They combine an operation with assignment.

Operator	Name	Description	Example	Explanation	Result
=	Simple Assignment	Assigns the value on the right to the variable on the left.	`int a = 10;` `a = 25;`	a gets the value 25	a = 25
+=	Add and Assign	Adds right operand to the left operand and assigns the result to the left operand.	`int a = 10;` `a += 5;`	`a = a + 5` `a = 10 + 5`	a = 15
-=	Subtract and Assign	Subtracts right operand from left operand and assigns the result to the left operand.	`int a = 10;` `a -= 4;`	`a = a - 4` `a = 10 - 4`	a = 6
*=	Multiply and Assign	Multiplies left operand with right operand and assigns the result to the left operand.	`int a = 10;` `a *= 3;`	`a = a * 3` `a = 10 * 3`	a = 30
/=	Divide and Assign	Divides left operand by right operand and assigns the result to the left operand.	`int a = 20;` `a /= 4;`	`a = a / 4` `a = 20 / 4`	a = 5
%=	Modulus and Assign	Computes modulus of left operand by right operand and assigns the result to the left operand.	`int a = 20;` `a %= 6;`	`a = a % 6` `a = 20 % 6`	a = 2

Also called as compounded assignment operators.

TERNARY OPERATOR IN JAVA (Conditional Operator)

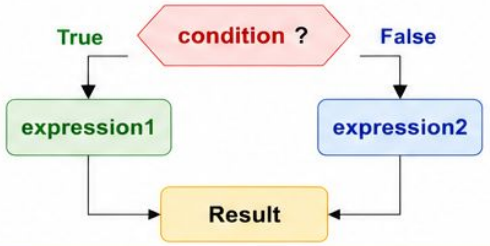
The ternary operator is a shorthand way of writing an if-else statement.

It takes three operands and has the form: `condition ? expression1 : expression2`

Part	Meaning	Description
condition	Test Expression	**Evaluated first.** Must be a boolean expression.
expression1	Value if condition is true	Returned (or evaluated) if condition is true.
expression2	Value if condition is false	Returned (or evaluated) if condition is false.

How it works?

If condition is true,
the result of expression1 is returned.
Otherwise (condition is false),
the result of expression2 is returned.



EXAMPLES

#	Example	Explanation	Result

1	`int a = 10, b = 20;`
	`int max = (a > b) ? a : b;`
	`System.out.println(max);`
		Check if a > b is true?
		Yes (false). So, value of b is returned.
			20

2	`int a = 15, b = 10;`
	`int max = (a > b) ? a : b;`
	`System.out.println(max);`
		Check if a > b is true?
		Yes (true). So, value of a is returned.
			15

3	`int n = -5;`
	`String msg = (n >= 0) ? "Positive" : "Negative";`
	`System.out.println(msg);`
		Check if n >= 0 is true?
		No (false). So, "Negative" is returned.
			Negative

4	`int mark = 75;`
	`String result = (mark >= 40) ? "Pass" : "Fail";`
	`System.out.println(result);`
		Check if mark >= 40 is true?
		Yes (true). So, "Pass" is returned.
			Pass

5	`int x = 7;`
	`int evenOdd = (x % 2 == 0) ? 1 : 0;`
	`System.out.println(evenOdd);`
		Check if x % 2 == 0 is true?
		No (false). So, 0 is returned.
			0

Equivalent if-else Statements

```
int max = (a > b) ? a : b;
```
is same as
```
int max;
if (a > b)
    max = a;
else
    max = b;
```

```
String msg = (n >= 0) ?
"Positive" : "Negative";
```
is same as
```
String msg;
if (n >= 0)
    msg = "Positive";
else
    msg = "Negative";
```

```
int evenOdd = (x % 2 == 0) ? 1 : 0;
```
is same as
```
int evenOdd;
if (x % 2 == 0)
    evenOdd = 1;
else
    evenOdd = 0;
```

Note: Ternary operator is right associative. Use parentheses for complex conditions for better readability.

Points to Remember

- ✓ It improves code readability for simple if-else conditions.
- ✓ Not recommended for complex logic.

TYPE COMPARISON OPERATOR IN JAVA



The **instanceof** operator is used to compare an object with a type (class or interface). It returns **true** if the object is an instance of that type; otherwise it returns **false**.

Operator	Name	Description	Syntax	Example	Result

instanceof	Type Comparison Operator	Checks whether an object is an instance of the specified type.	`object instanceof Type`	`String s = "Hello";`
				`boolean b = s instanceof String;`
				`System.out.println(b);`
					true

EXAMPLES

1

```
Object obj = new String("Java");
boolean r = obj instanceof String;
System.out.println(r);   // true
```

obj refers to a String object.
So, it is an instance of **String**.

2

```
Object obj = new Integer(10);
boolean r = obj instanceof Number;
System.out.println(r);    // true
```

Integer is a subclass of **Number**.
So, it is an instance of **Number**.

3

```
String s = "Java";
boolean r = s instanceof Integer;
System.out.println(r);    // false
```

String is not an instance of **Integer**.
So, result is false.

Points to Remember

- Used with reference variables (objects).
- Used to check the type before type casting.
- Returns boolean (**true**/**false**).

`object instanceof Type`
↓
true or **false**

Precedence and Associativity of Operators

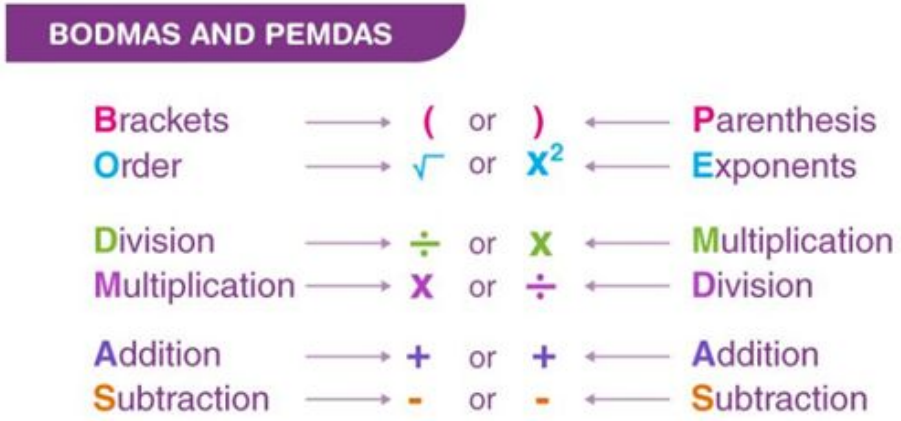
- **Precedence** and **associativity** are two features of Java operators.
- When there are two or more operators in an expression, the operator with the highest priority will be executed first.
  - Ex: 1 + 2 * 5
  - Precedence specifies – Operator with highest priority level
- What if the precedence of all operators in an expression is the same?

  Ex: 6*4%5 - Precedence is same at same level

  Ans: Associativity comes into the existence
- **Associativity** specifies the order in which operators are executed, which can be left to right or right to left.

```
int a = b = c = 8 ;   // right to left
```

- **BODMAS** rule is a rule in mathematics used to solve the expression involving more operators.





O – Order of means – powers, Square root, exponents

….

Precedence	Operator	Type	Associativity
1 (Highest)	() [] .	Parentheses Array subscript Member selection	Left to Right
2	++ --	Unary post-increment Unary post-decrement	Left to Right
3	++ -- + - ! ~ (type)	Unary pre-increment Unary pre-decrement Unary plus Unary minus Unary logical negation Unary bitwise complement Unary type cast	Right to left
4	* / %	Multiplication Division Modulus	Left to right
5	+ -	Addition Subtraction	Left to right
6	<< >> >>>	Bitwise left shift Bitwise right shift with sign extension Bitwise right shift with zero extension	Left to right

Precedence	Operator	Type	Associativity
7	< <= > >= instanceof	less than less than or equal greater than greater than equal Type comparison (objects only)	Left to right
8	== !=	Relational is equal to Relational is not equal to	Left to right
9	&	Bitwise AND	Left to right
10	^	Bitwise exclusive OR	Left to right
11	|	Bitwise inclusive OR	Left to right
12	&&	Logical AND	Left to right
13	||	Logical OR	Left to right
14	? :	Ternary conditional	Right to left
15 (lowest)	= += -= *= /= %=	Assignment Addition assignment Subtraction assignment Multiplication assignment Division assignment Modulus assignment	Right to left

Assignment Operator

- The associativity of = is from *right to left*.
- On the left of =, we write the name of the variable to which a value is to be assigned, and on its right, we write the value to be assigned.
- The name or identifier on the left is also called ***l*-value** and the value on the right of = is called ***r*-value**.

l-value(name of the identifier) = r-value(identifier or value)

Chain of Assignments:

```
int a,b,c,d;
A=b=c=d=50;
```

Basic Arithmetic Operators

Table 3.7 Basic arithmetic operators

Operation	*Operator*	*Description*
Addition	+	Adds two numbers (binary), changes sign (unary)
Subtraction	–	Subtracts two numbers (binary), changes sign (unary)
Multiplication	*	Multiplies two operands (binary)
Division	/	Divides an operand by another (binary)
Modulus	%	Gives remainder of a division operation. In Java, it can be applied to both integers as well as floating point numbers.

Ex:ArithmeticOperators.java

Combinational Assignment Operators

- In a compound assignment operator, the operations by an operator and assignment are combined in a single operator. As for e.g.
- int n = 8, m = 10;
- n + = m;

Program 3.23: Illustration of combinational operators

```
 1  class Operators2{
 2  public static void main (String args[]){
 3  int a = 4, b = 8, c= 9, d = 12;
 4  int e = 7, f = 10;
 5  b += a;
 6  f -= e;
 7  d /= 3;
 8  c %= a;
 9  e *= 3;
10  System.out.println ("b = " + b );
11  System.out.println ("c = " + c + "\t d = " + d);
12  System.out.println ("e = " + e + "\t f = " + f);
13  }}
```

Ternary Operator

- Ternary operator is also called conditional operator.
- In Java, there is only one ternary operator described by token (?:).
- It takes three operands for its operation.

expression1? expression2 : expression3

- If expression1 is true, then expression2 is executed; otherwise, expression3 is executed.

Relational Operators

Table 3.10 Relational operators

Operator	*Description*	*Code example with if (expression)*
==	Equality	`if (a == b)`
!=	Non-equality	`if (a != b)`
>	Greater than	`if (a > b)`
<	Less than	`if (a < b)`
>=	Greater or equal to	`if (a >= b)`
<=	less or equal to	`if (a <= b)`

Boolean Logical Operators

Table 3.11 Boolean logical operators

Operator	*Description*	*Description*
&	AND	It is true only when both operands in x&y are true.
|	OR	It is false when both operands in x|y are false.
!	NOT	If x = false, then !x = true and vice versa.
^	XOR	False when both x and y in expression x^y are either true or false; otherwise true.
||	OR	Short-circuit form of OR. In x||y, if x = true, then y is not evaluated because the value is anyway true.
&&	AND	Short-circuit form of AND. In x&&y, if x evaluates false, then y is not evaluated, because result is anyway false.

Bitwise Logical Operators

- These operators are applied to individual bits of numbers.
- The bitwise operators are OR, AND, XOR, and bitwise compliment/bitwise NOT operator.

Table 3.14 Bitwise logical operators

Operator	*Description*	*Operator*	*Description*
~	Bitwise complement unary NOT	>>	Right shift filled with previous content called *sign shift.*
|	Bitwise OR	<<	Left shift
&	Bitwise AND	>>=	Right shift assign
^	Bitwise XOR	<<=	Left shift assign
|=	Bitwise OR and assign	>>>	Right shift with zero fill on left
&=	Bitwise AND and assign	>>>=	Right shift, zero fill and assign
^=	Bitwise XOR and assign		

Bitwise Logical Operators

- These operators are applied to individual bits of numbers.
- The bitwise logical operators are AND(&), OR(|), XOR(^), and NOT(~)

Bitwise *OR* (|)

```
0110   - 6
0101    -5
-----
0111     - 7
```

Bitwise *AND* (&)

```
0110  - 6
0101    -5
-----
0100  -  4
```

Bitwise XOR (^)

```
0110  - 6
0101 - 5
------
0011  -3
```

Bitwise COMPLEMENT (~)

```
0110 ->   1001
```

Shift operators: << - left shift , >> right shift

Syntax: variable << or >> number_of_digits_to_be_shift

Ex: Bitwise_Operators.java

- **Bitwise Shift Operators**
- Syntax:

  value <operator> <number_of_times>

  - The left side of the expression is the integer that is shifted, and the right side of the expression denotes the number of times that it has to be shifted.
  - Bitwise shift operators are further classified as bitwise left and bitwise right shift operators.

Formatted output with printf() method

- Java provides two ways to format the output displayed on the screen:
  1. **Formatter Class**
  2. **printf() Method** (Most commonly used)
- Formatting means:
  - Right and left justification of text.
  - Controlling the precision of floating-point numbers
  - Displaying integers in decimal, octal, and hexadecimal forms
  - Displaying date and time in different formatsSyntax:
  - Controlling field width and alignment

System.out.printf("Formatting String", variables separated with comma);
System.out.printf("Price :%d",22,"Rupees");

JAVA printf() FORMAT SPECIFIERS

Used in System.out.printf() or System.out.format() to format the output.

Type Variable	Conversion Letter	Formatting String	Description
Integer	d	%d	Decimal (base 10) integer
Integer (Octal)	o	%o	Octal (base 8) integer
Integer (Hexadecimal)	x or X	%x or %X	Hexadecimal (base 16) integer
Character	c (or C)	%c (or %C)	Character (%C prints uppercase of the character)
String	s (or S)	%s (or %S)	String
Floating-point number	f (or F)	%f (or %F)	Decimal floating-point number
Floating-point number (Exponential form)	e (or E)	%e (or %E)	Exponential (scientific) notation
Boolean	b (or B)	%b (or %B)	Boolean value (true / false)
New Line	n	%n	Prints a new line
Percent Sign	%	%%	Prints the % symbol

Examples

- System.out.printf("%d", 25); → 25
- System.out.printf("%x", 255); → ff
- System.out.printf("%.2f", 12.3456); → 12.35
- System.out.printf("%s", "Java"); → Java
- System.out.printf("%e", 123.45); → 1.234500e+02
- System.out.printf("%b", true); → true
- System.out.printf("Hello%nWorld"); → Hello World
- System.out.printf("Discount: 50%% off"); → Discount: 50% off

Ex: O/p of diff types of variables – Format_String.java

Formatting output of integer numbers:

Feature	Description	Syntax	Example
Decimal Integer	Prints the integer in decimal form.	%d	System.out.printf("%d", 163);
Field Width	Specifies the minimum number of character positions reserved for the integer.	%nd	System.out.printf("%10d", 163);
Right Justification	Default alignment. The integer is right-aligned within the specified field width.	%10d	`
Left Justification	Use - after % to left-align the integer.	%-10d	`
Octal Format	Displays the integer in octal (base 8).	%o	243
Hexadecimal Format	Displays the integer in hexadecimal (lowercase).	%x	a3
Hexadecimal (Uppercase)	Displays the integer in hexadecimal (uppercase).	%X	A3

Formatting output of Floating point numbers:

- Field width – the field width is the integer number placed between % sign and conversion character **F or f**
- Ex: Formatting_Floating.java

Format	**Description**
%f	Floating-point number (6 decimal places by default)
%10f	Right-justified in a field of width 10
%-10f	Left-justified in a field of width 10
%.2f	Displays 2 digits after the decimal point
%10.2f	Field width 10 with 2 decimal places

Formatting output of Strings

- String formatting in Java is done using the **%s** or **%S** conversion specifier with the printf() method.

Format	Description	Example Output
%s	Prints the string	Java
%S	Prints the string in uppercase	JAVA
%10s	Right-justified in 10 spaces	Java
%-10s	Left-justified in 10 spaces	Java
%.4s	Prints first 4 characters	Java
%10.4s	Width 10, prints first 4 characters	Java

- Ex: Formatting_String.java

Assignment Questions

- Interpret the differences between local, instance and static(class) variables with suitable examples.
- Write a JAVA program to print factorial of a given number
- Conversion of Decimal to Binary
- Write a JAVA program to generate int and double random numbers using random() method of Math class.
- JAVA program to calculate area of a triagle.
- Develop a program to generate random of 8 integer numbers and calculate its average.

Control Statements

- **Sequential Control Structure:**Statements are executed one after another in the order they appear.
- ***Selection statements:*** These statements give an option of choosing different paths of execution of a program depending on the result of a test condition.
  - The expressions in this group are *if*, *if–else*, and *switch statement*.
- ***Iteration statements:*** These statements allow repeated evaluation of a statement or a block of statements if some condition is satisfied.
  - This group contains statements : *while*, *do–while*, *for*, and *for– each*.
- ***Jump statements:*** These statements make the program control jump to another part of a program in the forward or backward directions with or without a test condition.
  - The statements in this group are *break*, *break label*, *continue*, *continue label*, *and return*.

if Statement

- The code for *if* statement is as follows:

```
if (conditional expression)
        statement;
```

Program 4.1: Illustration of the application of *if* (expression)

```
1    class IfExp {
2     public static void main (String args[])
3    {
4    int n = 51 ;  // integer variable
5     if(n %2 == 0)  // modulus division and checking the remainder
6    System.out.println ("n is an even number");
7     if (n %2 != 0)
8    System.out.println ("n is an odd number.");
9     }
10   }
```

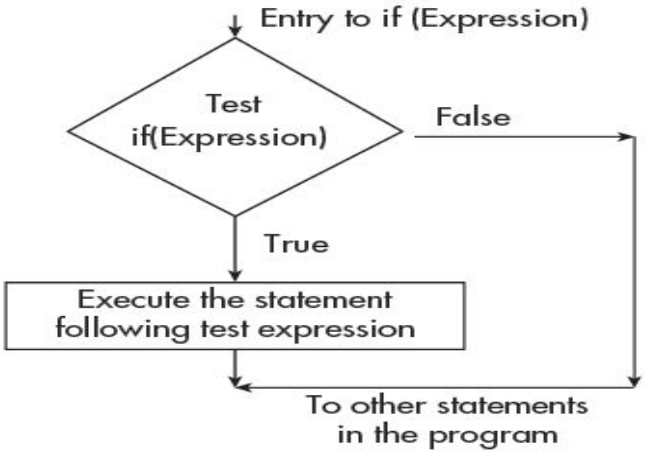
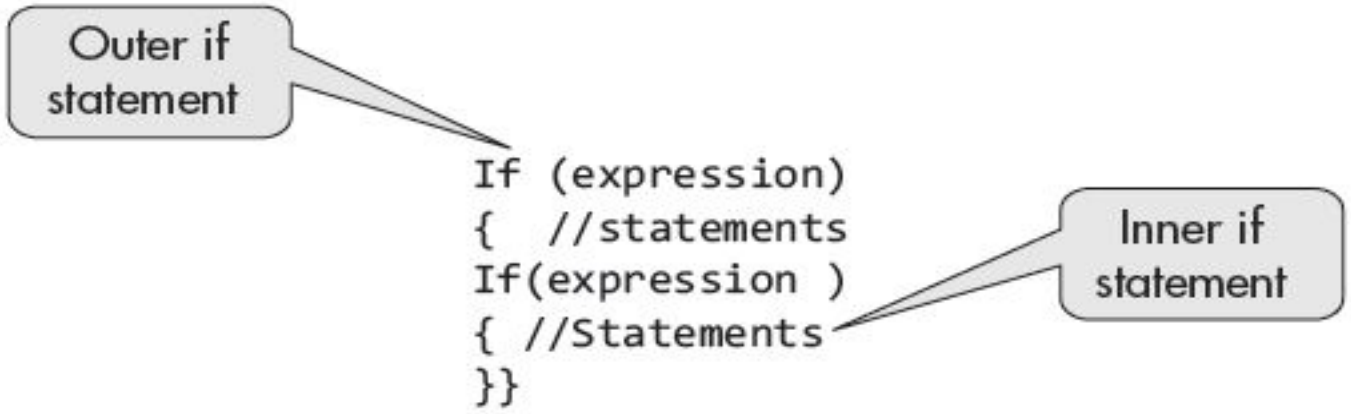


Fig. 4.1 Working of *if* (expression)

Nested if Expressions



If –else statement

- The code for *if –else* statement is as:

```
if (expression)
{
    /* statements */
}
else {
    /* statements */
}
```

Working

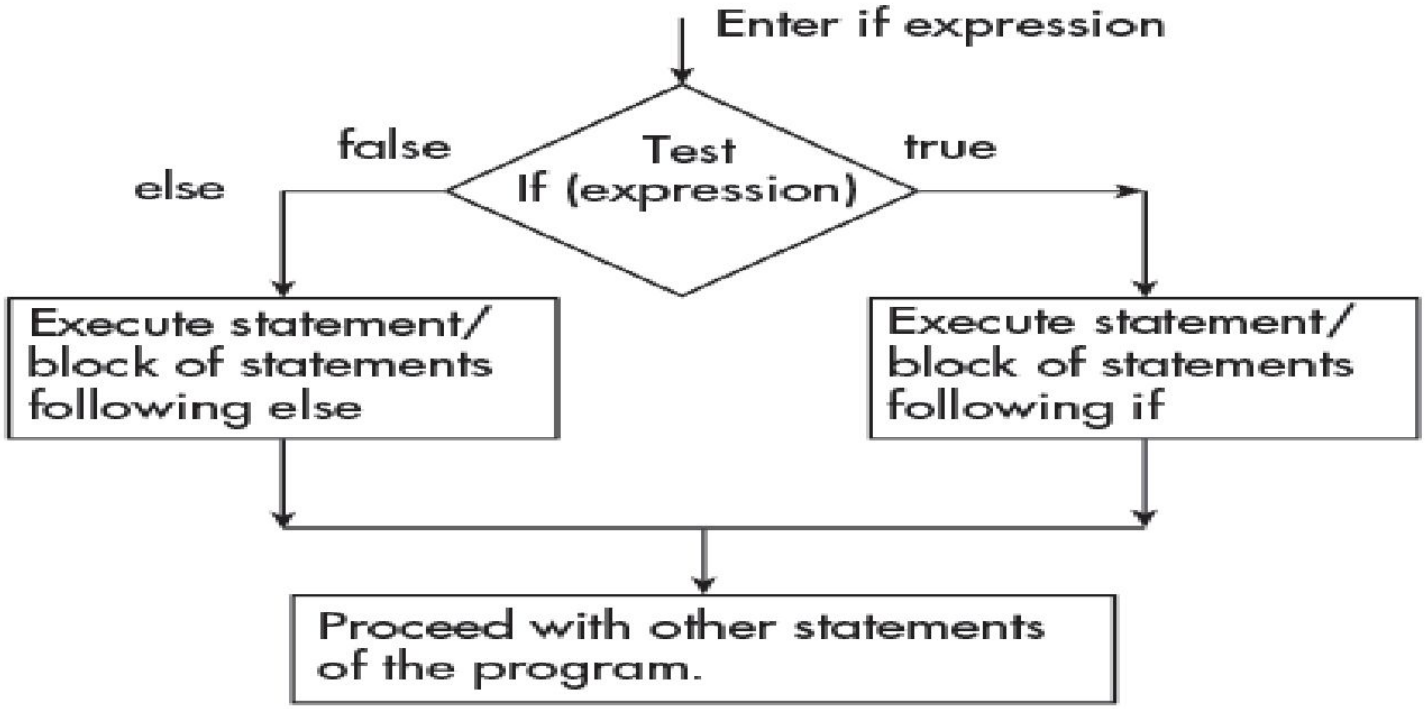


Fig. 4.2 Working of *if–else* expressions

Else-if ladder - if-else chain

if(*condition)*
statement;
else if(*condition)*
statement;
else if(*condition)*
statement;
.
.
.
else
statement;

The **if statements are executed from the top down. As soon as one of the conditions** controlling the **if is true, the statement associated with that if is executed, and the rest of** the ladder is bypassed. If none of the conditions is true, then the final **else statement will be** executed.

Ternary Operator ?:

- Its code is as follows:

  Expression1 ?Expression2 :Expression3
- The first expression is the test condition.
- If it evaluates true, the Expression2 is executed; otherwise Expession3 is executed

Switch Statement

```
switch (expression)
{
    case value1:
      // statement sequence
      break;
case value2:
      // statement sequence
      break;
      .
    case valueN :
      // statement sequence
      break;
default:
    // default statement sequence
}
```

1. For versions of Java prior to JDK 7, *expression must be of type* ***byte, short, int, char, or an*** enumeration.
2. Beginning with JDK 7, *expression* can also be of type **String.**

- The switch expression is evaluated once.
- The value of the expression is compared with the values of each case.
- If there is a match, the associated block of code is executed.
- The break and default keywords are optional

Nested Switch Statement

```
switch(count)
{
    case 1:
      switch(target) { // nested switch
      case 0:
            System.out.println("target is zero");
            break;
      case 1: // no conflicts with outer switch
            System.out.println("target is one");
            break;
    }
    break;
    case 2:
    .
    .
    .
```

while

```
while(condition)
{
    // body of loop
}
```

```
do {
     // body of loop
}while(condition);
```

```
class WhileSt {
    public static void main(String args[])
    {
        int n = 10;
        while(n > 0) {
            System.out.println("tick " + n);
            n--;
        }
    }
}
```

```
class DoWhileSt
{
    public static void main(String args[]) {
        int n = 10;
        do {
            System.out.println("tick " + n);
            n--;
        } while(n > 0);
    }
}
```

while Expression

- The while loop may be coded as:

  while (test expression)

  statement;

- If the *while* test expression evaluates true, the statement following the expression is executed.

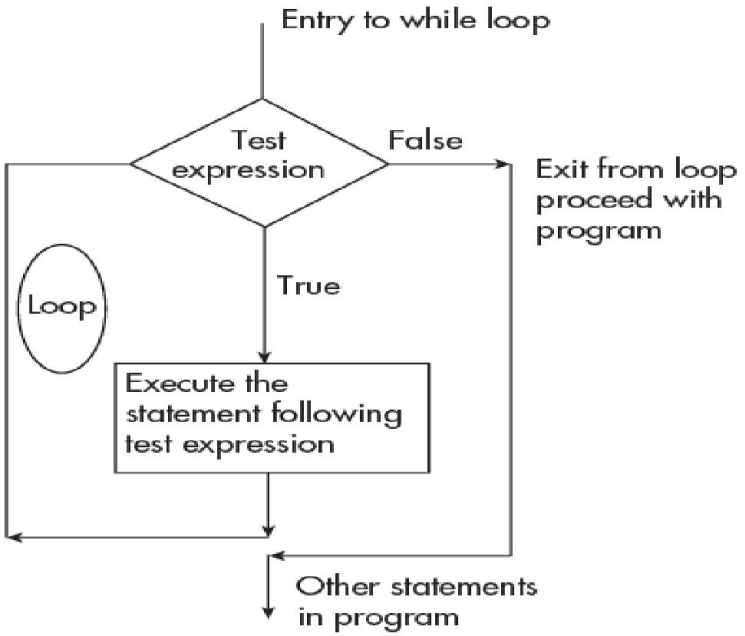


Fig. 4.5 Working of *while* loop

Program 4.8: Illustration of while loop

```
1   class WhileExp {
2    public static void main (String args[]) {
3   int n = 1, sum = 0;
4   while (n <=10) // testing the condition
5    {sum += n; // computing the sum
6    n = n + 1;} // increments the value of variable
7   System.out.println ("sum = " + sum);
8    }
9   }
```

do while Loop

- This loop is similar to *while* loop except for the fact that the test expression is placed at the end.
- The statement to be executed is placed between *do* and *while* expression.

  Code is:

```
do
        Statement;
while (test expression);
```

do while Loop

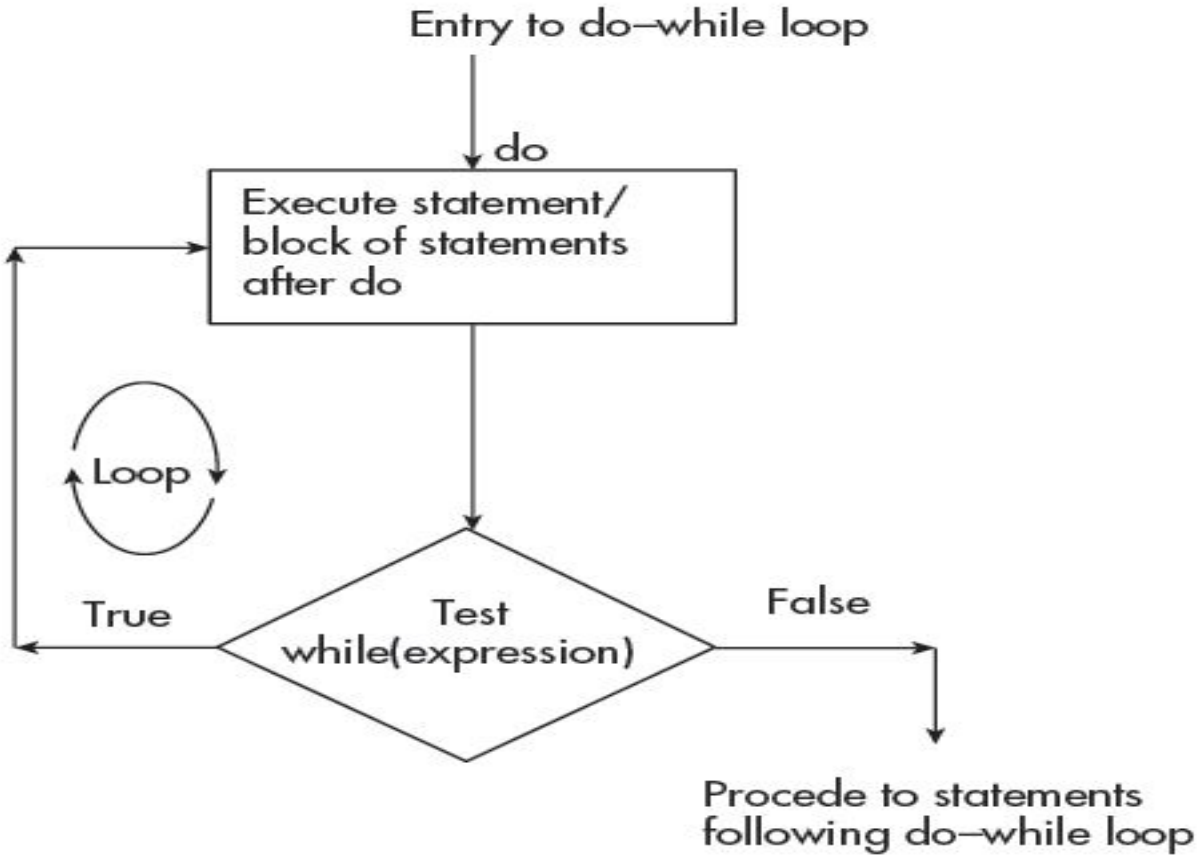


Fig. 4.6 Working of *do–while* loop

© Oxford University Press 2018. All rights reserved.

endless while loop

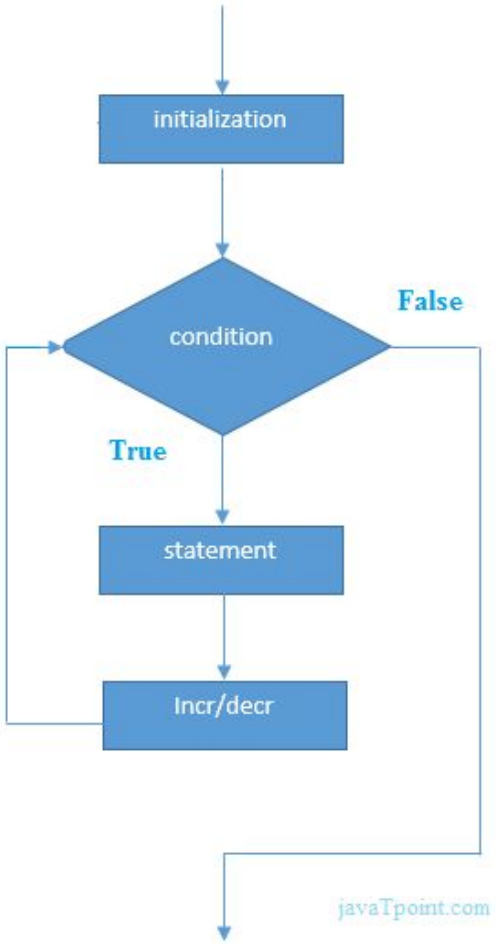
- The endless while loop may be coded as:

```
while (true) {
    // do something
}
 do {
    // do something
} while(true);
```

For loop

- The *for* loop is the most versatile loop when the start and end points of the loop are known.
- It is an entry controlled loop

```
for(initialization; condition;
   incr/decr)
{
   //code to be executed
}
```



Nested for loop

```
for (int i = 0; i < 8; i++)
    { statement;                          // Outermost for loop
      ---------
  for(int j = 0; j < 10; j++)
       { statement;                       // Middle for loop
         ---------
      for (int k = 0; k < 12; k++)
            { statement;                  //innermost
              --------                    // for loop
               --------    }
               statement;
                -------    }
                statement;
                --------
                --------
                           }
```

Fig. 4.8 Nested *for* loops

The for-each loop

- The for-each loop is used to traverse array or collection in java. It is easier to use than simple for loop because we don't need to increment value and use subscript notation.
- It works on elements basis not index. It returns element one by one in the defined variable.
- Syntax:

```
for(data_type var:array)
{
 //code to be executed
}
```

```
public class ForEachExample
{
  public static void main(String[] args)
{
            int arr[]={12,23,44,56,78};
            for(int i:arr)
            {
                System.out.println(i);
            }
       }
}
```

Nested for-each for loop

```
public class NestedForEach
{
   public static void main(String[] args)
   {
      String[][] str = {{"Amaravati","Vij","Gnt"}, {"Hyd","WL","Khammam"}};

      for (String[] s : str)
      {
         for(String c:s)
         {
       System.out.println(c);
          }
      }
   }
}
```

Break Statement

- This is used for exiting from a loop and control goes to the end of the current loop.

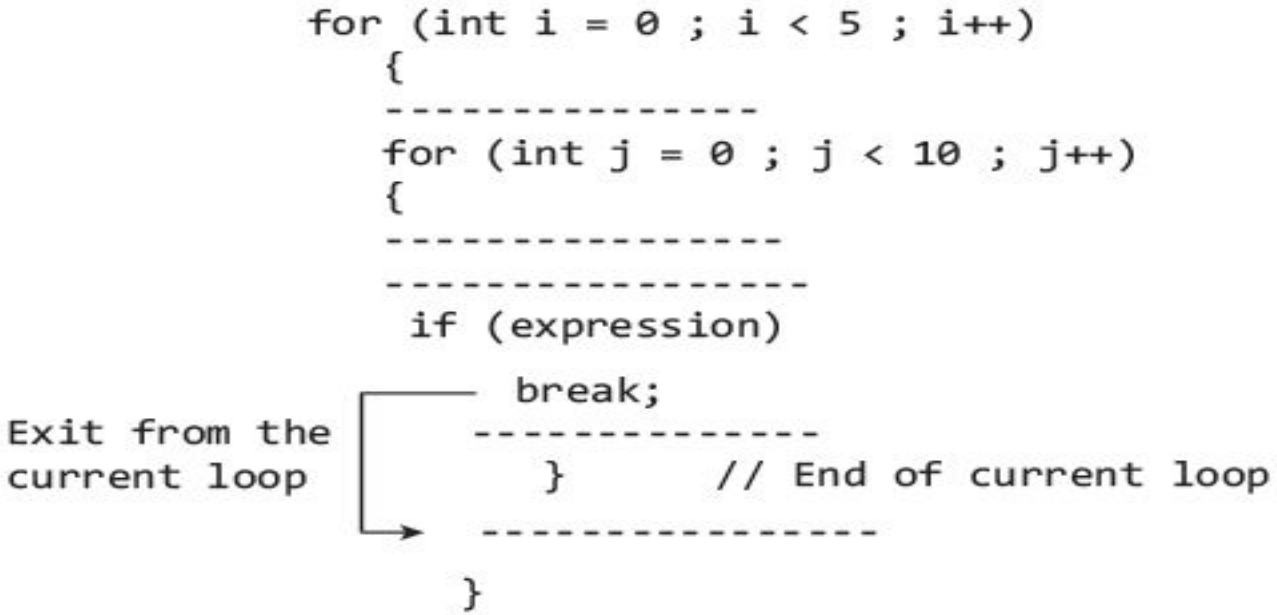
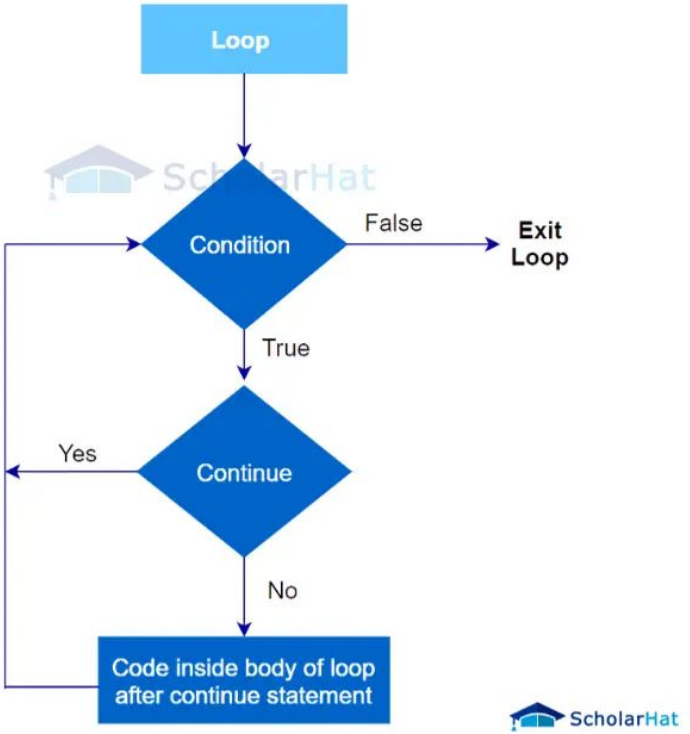


Fig. 4.9 Illustration of *break;* statement

Continue statement

- The continue statement is used to skip the current iteration of any loop and bring the control to the beginning of the iteratic



JUMP STATEMENTS WITH LABEL

Labels can be used with break or continue to transfer control to the labeled outer loop.

3. break with label

```
outer:
  for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
      if (j == 2)
          break outer;  // exit outer loop completely
      System.out.println("i = " + i + ", j = " + j);
    }
  }
```

How it works

When `j == 2`, `break outer;` terminates the labeled outer loop immediately.

4. continue with label

```
outer:
  for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
      if (j == 2)
          continue outer;  // skip to next i
      System.out.println("i = " + i + ", j = " + j);
    }
  }
```

How it works

When `j == 2`, `continue outer;` skips the remaining statements of the current outer iteration and moves to the next value of `i`.

Key Points

- **break label;**
  - Terminates the labeled loop (outer loop here).
  - Control moves to the statement after the labeled loop.
- **continue label;**
  - Skips the remaining part of the current iteration of the labeled loop.
  - Control moves to the next iteration of the labeled loop.

Break Statement with Label

- Break with a label statement is similar to goto statement in C++, but the implementation is different.

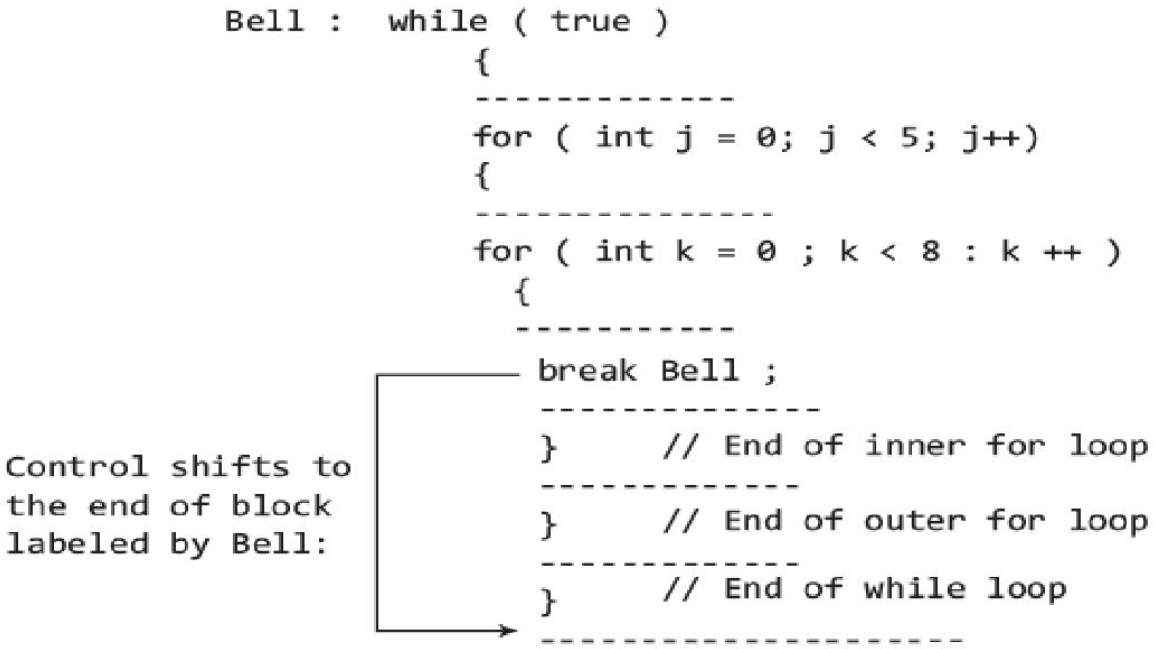


Fig. 4.10 Illustration of break statement with a label

Continue with label

- We can **use** a **labelled continue statement** to **continue** the outermost loop.

Questions

- Diff between while, do-while and for loops
- Diff between break and continue
- To check the given number is even or odd
- To check the given number is prime or not
- Even numbers – with in a specific range
- Prime numbers with in a specific range
- Printing Fibonacci numbers up to n.
- To check the given number is palindrome or not
- To check whether the given number is Armstrong or not.(an Armstrong number is a number that equals the sum of its individual digits, each raised to the power of the number of digits.)